==

Scholay学术平台 V1.0 源代码

（软件著作权登记材料）

==

软件名称：Scholay学术平台
版本号：V1.0
著作权人：成都恩特罐罐文化科技有限公司
统一社会信用代码：91510100MAEEHRNE26

Copyright (c) 2025 成都恩特罐罐文化科技有限公司
All Rights Reserved. 保留所有权利。

说明：
本文档为软件著作权登记用源代码，按照登记要求提供：
- 前30页：源代码前1500行（每页50行）
- 后30页：源代码后1500行（每页50行）

==
第一部分：前30页源代码
==

```
// ============================================================================
// Scholay学术平台 V1.0
// Copyright (c) 2025 成都恩特罐罐文化科技有限公司
// 统一社会信用代码: 91510100MAEEHRNE26
// All Rights Reserved.
// ============================================================================


// ========== 文件: backend/cmd/server/main.go ==========
// ============================================================================
// Scholay学术平台 V1.0
// Copyright (c) 2025 成都恩特罐罐文化科技有限公司
// 统一社会信用代码: 91510100MAEEHRNE26
// All Rights Reserved.
// ============================================================================

package main

import (
 "context"
 "log"
 "os"
 "os/signal"
 "syscall"
 "time"
 "scholar-backend/internal/admin"
 "scholar-backend/internal/ai"
 "scholar-backend/internal/feedback"
 "scholar-backend/internal/common/cache"
 "scholar-backend/internal/common/cleanup"
```

```
  31  appconfig "scholar-backend/internal/common/config"
  32  "scholar-backend/internal/common/database"
  33  "scholar-backend/internal/common/middleware"
  34  "scholar-backend/internal/common/telemetry"
  35  "scholar-backend/internal/config"
  36  "scholar-backend/internal/conversation"
  37  "scholar-backend/internal/coupon"
  38  "scholar-backend/internal/mcp"
  39  "scholar-backend/internal/order"
  40  "scholar-backend/internal/payment"
  41  "scholar-backend/internal/peerreview"
  42  "scholar-backend/internal/proxy"
  43  "scholar-backend/internal/semanticscholar"
  44  "scholar-backend/internal/subscription"
  45  "scholar-backend/internal/thirdparty"
  46  "scholar-backend/internal/user"
  47  "scholar-backend/internal/wechat"
  48  "scholar-backend/internal/wechatofficial"
  49
  50  "github.com/gin-gonic/gin"
  51  "go.opentelemetry.io/otel/attribute"
  52 )
  53
  54 func main() {
  55  // Load configuration
  56  cfg := appconfig.Load()
  57
  58  // Initialize OpenTelemetry with SigNoz
  59  ctx := context.Background()
  60  if cfg.SigNoz.EnableTracing || cfg.SigNoz.EnableLogging {
  61   telemetryCfg := &telemetry.Config{
  62    Endpoint:      cfg.SigNoz.Endpoint,
  63    ServiceName:   cfg.SigNoz.ServiceName,
  64    Environment:   cfg.SigNoz.Environment,
  65    EnableTracing: cfg.SigNoz.EnableTracing,
  66    EnableLogging: cfg.SigNoz.EnableLogging,
  67    EnableMetrics: cfg.SigNoz.EnableMetrics,
  68    SamplingRate:  cfg.SigNoz.SamplingRate,
  69   }
  70
  71   // Set global endpoint for logger
  72   telemetry.SetGlobalEndpoint(telemetryCfg.Endpoint)
  73
  74   telemetryShutdown, err := telemetry.Initialize(ctx, telemetryCfg)
  75   if err != nil {
  76    log.Printf("Warning: Failed to initialize telemetry: %v", err)
  77   } else {
  78    log.Printf(" Telemetry initialized (SigNoz endpoint: %s)", telemetryCfg.Endpoint)
  79    log.Printf("  - Service: %s", telemetryCfg.ServiceName)
  80    log.Printf("  - Environment: %s", telemetryCfg.Environment)
  81    log.Printf("  - Tracing: %v, Logging: %v", telemetryCfg.EnableTracing, telemetryCfg.Enable
Logging)
  82    log.Printf("  - Sampling Rate: %.1f%%", telemetryCfg.SamplingRate*100)
  83
  84    defer func() {
  85     shutdownCtx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
  86     defer cancel()
```

```
    if err := telemetryShutdown(shutdownCtx); err != nil {
     log.Printf("Error shutting down telemetry: %v", err)
    }
   }()
  }
 } else {
  log.Println(" Telemetry disabled by configuration")
 }

 // Create logger for main (only if telemetry is enabled)
 var logger *telemetry.Logger
 if cfg.SigNoz.EnableLogging {
  logger = telemetry.NewLogger("main")
  logger.Info(ctx, "Starting Scholar Backend Service",
   attribute.String("version", "1.0.0"),
   attribute.String("environment", cfg.SigNoz.Environment),
  )
 }

 // Initialize Redis
 if err := cache.InitRedis(cfg.Redis.Host, cfg.Redis.Port, cfg.Redis.Password, cfg.Redis.DB);
err != nil {
  if logger != nil {
   logger.Fatal(ctx, "Failed to initialize Redis", err)
  }
  log.Fatalf("Failed to initialize Redis: %v", err)
 }
 if logger != nil {
  logger.Info(ctx, "Redis initialized successfully")
 }
 log.Println(" Redis initialized successfully")

 // Initialize cache worker pool (prevents goroutine accumulation from async cache writes)
 cache.InitWorkerPool(10, 100) // 10 workers, queue size 100

 // Initialize database
 if err := database.Initialize(cfg); err != nil {
  log.Fatalf("Failed to initialize database: %v", err)
 }

 // Initialize cleanup service
 cleanupService := cleanup.NewCleanupService(database.GetDB())
 cleanupService.Start()

 // Initialize admin module
 adminRepo := admin.NewRepository(database.GetDB())
 adminService := admin.NewService(adminRepo)
 adminHandler := admin.NewHandler(adminService)

 // Initialize dashboard handler
 dashboardHandler := admin.NewDashboardHandler(database.GetDB())

 // Initialize feedback module
 feedbackRepo := feedback.NewRepository(database.GetDB())
 feedbackService := feedback.NewService(feedbackRepo)
 feedbackHandler := feedback.NewHandler(feedbackService, "./uploads")

```

```
  // Initialize default admin user
  if err := adminService.InitializeDefaultAdmin(cfg.Admin.InitialUsername, cfg.Admin.InitialPa
ssword); err != nil {
   log.Printf("Warning: Failed to initialize default admin: %v", err)
  } else {
   log.Printf("Default admin user initialized (username: %s)", cfg.Admin.InitialUsername)
  }

  // Initialize user module
  userRepo := user.NewRepository(database.GetDB())
  userService := user.NewService(userRepo)
  userHandler := user.NewHandler(userService, database.GetDB())

  // Initialize config module
  configRepo := config.NewRepository(database.GetDB())
  configService := config.NewService(configRepo)
  configHandler := config.NewHandler(configService)

  // Initialize coupon module
  baseURL := "http://localhost:" + cfg.Server.Port // TODO: Configure from env
  redisClient := cache.GetRedis()
  couponService := coupon.NewService(database.GetDB(), userRepo, redisClient, baseURL)
  couponHandler := coupon.NewHandler(couponService)

  // Start coupon scheduler for automated tasks
  couponScheduler := coupon.NewScheduler(couponService)
  couponScheduler.Start()

  // Inject config service into user service (for email sending)
  userService.SetConfigService(configService)

  // Inject config service into feedback service (for reply email sending)
  feedbackService.SetConfigService(configService)

  // Inject user repository into feedback service (for user info lookup)
  feedbackService.SetUserRepository(&feedbackUserRepoAdapter{userRepo: userRepo})

  // Inject coupon service into user service (for invitation handling)
  userService.SetCouponService(couponService)

  // Initialize default privacy policy
  if err := configService.InitializeDefaultPrivacyPolicy(); err != nil {
   log.Printf("Warning: Failed to initialize default privacy policy: %v", err)
  } else {
   log.Printf("Default privacy policy initialized")
  }

  // Initialize default terms of service
  if err := configService.InitializeDefaultTermsOfService(); err != nil {
   log.Printf("Warning: Failed to initialize default terms of service: %v", err)
  } else {
   log.Printf("Default terms of service initialized")
  }

  // Initialize default system config
  if err := configService.InitializeDefaultSystemConfig(); err != nil {
   log.Printf("Warning: Failed to initialize default system config: %v", err)
```

```
199 } else {
200  log.Printf("Default system config initialized")
201 }
202
203 // Initialize Third-party Account Management
204 thirdPartyRepo := thirdparty.NewRepository(database.GetDB())
205 thirdPartyService := thirdparty.NewService(thirdPartyRepo)
206 thirdPartyHandler := thirdparty.NewHandler(thirdPartyService)
207
208 // Start cache refresh task (every 5 minutes)
209 thirdPartyService.StartCacheRefreshTask(5 * time.Minute)
210 log.Printf("Third-party account service initialized with cache refresh")
211
212 // Initialize AI service (需要在scholar handler之前初始化)
213 aiService := ai.NewService(database.GetDB(), thirdPartyService)
214
215 // Initialize Semantic Scholar module with key rotation
216 scholarManager := semanticscholar.NewManager(thirdPartyService)
217 scholarHandler := semanticscholar.NewHandler(scholarManager, aiService) // 传入aiService用于查询翻译
218 mcpHandler := mcp.NewHandler(scholarManager) // MCP uses manager for dynamic API key access
219 aiHandler := ai.NewHandler(aiService)
220
221 // Initialize Peer Review module
222 mcpServer := mcp.NewMCPServer(scholarManager)
223 peerReviewService := peerreview.NewService(database.GetDB(), aiService, mcpServer, thirdPart
yService)
224 peerReviewHandler := peerreview.NewHandler(peerReviewService)
225
226 // Initialize Subscription module
227 subscriptionService := subscription.NewService(database.GetDB())
228 subscriptionHandler := subscription.NewHandler(database.GetDB())
229
230 // Initialize Conversation module
231 conversationRepo := conversation.NewRepository(database.GetDB())
232 conversationService := conversation.NewService(conversationRepo, scholarManager, aiService,
subscriptionService)
233 conversationHandler := conversation.NewHandler(conversationService)
234 chatHandler := conversation.NewChatHandler(conversationRepo, aiService, conversationService,
 subscriptionService)
235
236 // Initialize Proxy module
237 proxyHandler := proxy.NewHandler()
238
239 // Initialize Order module
240 orderService := order.NewService(database.GetDB())
241 orderHandler := order.NewHandlerWithService(orderService)
242
243 // Initialize Payment module
244 alipayHandler := payment.NewAlipayHandler(database.GetDB())
245 wechatPayRepo := payment.NewWeChatPayRepository(database.GetDB())
246 wechatPayService := payment.NewWeChatPayService(wechatPayRepo)
247 wechatPayHandler := payment.NewWeChatPayHandler(wechatPayService, orderService)
248
249 // Initialize WeChat module
250 wechatService := wechat.NewService(database.GetDB())
251 wechatHandler := wechat.NewHandler(database.GetDB())
252
```

```
253  // Create adapter and inject into user handler
254  wechatAdapter := wechat.NewServiceAdapter(wechatService)
255  userHandler.SetWeChatService(wechatAdapter)
256
257  // Initialize WeChat Official Account module
258  wechatOfficialService := wechatofficial.NewService(database.GetDB(), aiService)
259  wechatOfficialHandler := wechatofficial.NewHandler(wechatOfficialService)
260
261  // Setup Gin router
262  router := gin.Default()
263
264  // Apply global middleware
265  router.Use(middleware.CORSMiddleware())
266
267  // Add telemetry middleware only if enabled
268  if cfg.SigNoz.EnableTracing {
269   // Add OpenTelemetry middleware for distributed tracing
270   router.Use(middleware.OpenTelemetryGinMiddleware(cfg.SigNoz.ServiceName))
271  }
272
273  if cfg.SigNoz.EnableLogging {
274   // Add comprehensive telemetry middleware for logging
275   router.Use(middleware.TelemetryMiddleware())
276  }
277
278  // Add rate limiting middleware (100 requests per second per IP)
279  // This prevents DoS attacks and ensures fair resource allocation
280  router.Use(middleware.RateLimitMiddleware(100, 1*time.Second))
281
282  // Add adaptive timeout middleware
283  // Automatically adjusts timeout based on endpoint type
284  router.Use(middleware.APITimeoutMiddleware())
285
286  // API routes
287  api := router.Group("/api/v1")
288  {
289   // Config routes (public)
290   configHandler.RegisterRoutes(api)
291
292   // Admin routes
293   adminHandler.RegisterRoutes(api)
294
295   // Admin protected routes
296   adminProtected := api.Group("/admin")
297   adminProtected.Use(middleware.AdminAuthMiddleware())
298   {
299    adminProtected.GET("/profile", adminHandler.GetProfile)
300    adminProtected.PUT("/password", adminHandler.UpdatePassword)
301    adminProtected.POST("/verify-password", adminHandler.VerifyPassword)
302    // Dashboard statistics
303    adminProtected.GET("/dashboard/statistics", dashboardHandler.GetDashboardStatistics)
304    // Configuration management
305    // System configuration
306    adminProtected.PUT("/privacy-policy", configHandler.UpdatePrivacyPolicy)
307    adminProtected.PUT("/terms-of-service", configHandler.UpdateTermsOfService)
308    adminProtected.GET("/email-config", configHandler.GetEmailConfig)
309    adminProtected.PUT("/email-config", configHandler.UpdateEmailConfig)
```

```
adminProtected.POST("/email-config/test", configHandler.TestEmailConfig)
adminProtected.GET("/sms-config", configHandler.GetSMSConfig)
adminProtected.PUT("/sms-config", configHandler.UpdateSMSConfig)
adminProtected.POST("/sms-config/test", configHandler.TestSMSConfig)
adminProtected.PUT("/system-config", configHandler.UpdateSystemConfig)
// GitHub OAuth configuration
adminProtected.GET("/github-config", configHandler.GetGithubOAuthConfig)
adminProtected.PUT("/github-config", configHandler.UpdateGithubOAuthConfig)
// User management
adminProtected.GET("/users/stats", userHandler.GetUserStatistics)
adminProtected.GET("/users", userHandler.GetUserList)
adminProtected.GET("/users/online", userHandler.GetOnlineUsers)
adminProtected.GET("/users/:id", userHandler.GetUserDetail)
adminProtected.DELETE("/users/:id", userHandler.DeleteUser)
adminProtected.POST("/users/:id/grant-subscription", orderHandler.AdminGrantSubscription)
// Statistics charts
adminProtected.GET("/statistics/users", userHandler.GetUserStatisticsChart)
adminProtected.GET("/statistics/online", userHandler.GetOnlineUserChart)
// Third-party account management
adminProtected.GET("/third-party-accounts", thirdPartyHandler.ListAccounts)
adminProtected.POST("/third-party-accounts", thirdPartyHandler.CreateAccount)
adminProtected.GET("/third-party-accounts/stats", thirdPartyHandler.GetStats)
adminProtected.POST("/third-party-accounts/refresh-cache", thirdPartyHandler.RefreshCache)
adminProtected.GET("/third-party-accounts/:id", thirdPartyHandler.GetAccount)
adminProtected.PUT("/third-party-accounts/:id", thirdPartyHandler.UpdateAccount)
adminProtected.DELETE("/third-party-accounts/:id", thirdPartyHandler.DeleteAccount)
adminProtected.POST("/third-party-accounts/:id/test", thirdPartyHandler.TestAccount)
// Prompt template management
adminProtected.GET("/prompt-templates", aiHandler.ListPromptTemplates)
adminProtected.POST("/prompt-templates", aiHandler.CreatePromptTemplate)
adminProtected.GET("/prompt-templates/:id", aiHandler.GetPromptTemplate)
adminProtected.PUT("/prompt-templates/:id", aiHandler.UpdatePromptTemplate)
adminProtected.DELETE("/prompt-templates/:id", aiHandler.DeletePromptTemplate)
// Analysis dimensions management (Admin only)
adminProtected.GET("/analysis/dimensions", aiHandler.AdminGetSystemDimensions)
adminProtected.PUT("/analysis/dimensions/:id", aiHandler.AdminUpdateSystemDimension)
// Subscription plan management
adminProtected.PUT("/subscription-plans/:id", subscriptionHandler.UpdatePlan)

// Order management (Admin only)
adminProtected.GET("/orders/statistics", orderHandler.GetOrderStatistics)
adminProtected.GET("/orders", orderHandler.AdminListOrders)

// Alipay payment configuration (Admin only)
adminProtected.GET("/payment/alipay/config", alipayHandler.GetActiveConfig)
adminProtected.GET("/payment/alipay/configs", alipayHandler.ListConfigs)
adminProtected.POST("/payment/alipay/config", alipayHandler.CreateConfig)
adminProtected.PUT("/payment/alipay/config/:id", alipayHandler.UpdateConfig)
adminProtected.DELETE("/payment/alipay/config/:id", alipayHandler.DeleteConfig)
adminProtected.POST("/payment/alipay/config/:id/activate", alipayHandler.SetActiveConfig)

// WeChat Pay configuration (Admin only)
adminProtected.GET("/payment/wechat/config", wechatPayHandler.GetActiveConfig)
adminProtected.GET("/payment/wechat/configs", wechatPayHandler.GetAllConfigs)
adminProtected.POST("/payment/wechat/config", wechatPayHandler.CreateConfig)
adminProtected.PUT("/payment/wechat/config/:id", wechatPayHandler.UpdateConfig)
adminProtected.DELETE("/payment/wechat/config/:id", wechatPayHandler.DeleteConfig)
```

```
  adminProtected.POST("/payment/wechat/config/:id/activate", wechatPayHandler.ActivateConfig
)

  // WeChat OAuth configuration (Admin only)
  adminProtected.GET("/wechat/config", wechatHandler.GetActiveConfig)
  adminProtected.GET("/wechat/configs", wechatHandler.ListConfigs)
  adminProtected.POST("/wechat/config", wechatHandler.CreateConfig)
  adminProtected.PUT("/wechat/config/:id", wechatHandler.UpdateConfig)
  adminProtected.DELETE("/wechat/config/:id", wechatHandler.DeleteConfig)
  adminProtected.POST("/wechat/config/:id/activate", wechatHandler.SetActiveConfig)

  // WeChat Official Account configuration (Admin only)
  adminProtected.GET("/wechat-official/config", wechatOfficialHandler.GetConfig)
  adminProtected.PUT("/wechat-official/config", wechatOfficialHandler.UpdateConfig)

  // Coupon management (Admin only)
  adminProtected.POST("/coupon-templates", couponHandler.CreateTemplate)
  adminProtected.GET("/coupon-templates", couponHandler.ListTemplates)
  adminProtected.GET("/coupon-templates/:id", couponHandler.GetTemplate)
  adminProtected.PUT("/coupon-templates/:id", couponHandler.UpdateTemplate)
  adminProtected.DELETE("/coupon-templates/:id", couponHandler.DeleteTemplate)
  adminProtected.POST("/coupon-templates/:id/activate", couponHandler.ActivateTemplate)
  adminProtected.POST("/coupon-templates/:id/pause", couponHandler.PauseTemplate)

  // Coupon distribution (Admin only)
  adminProtected.POST("/coupons/distribute", couponHandler.DistributeCoupons)
  adminProtected.GET("/coupons/distributed", couponHandler.AdminListAllDistributedCoupons)
  adminProtected.GET("/coupons/statistics", couponHandler.GetStatistics)

  // Batch distribution (Admin only)
  adminProtected.POST("/coupon-batches", couponHandler.CreateBatch)
  adminProtected.GET("/coupon-batches", couponHandler.ListBatches)
  adminProtected.GET("/coupon-batches/:id", couponHandler.GetBatch)
  adminProtected.POST("/coupon-batches/:id/execute", couponHandler.ExecuteBatch)
  adminProtected.POST("/coupon-batches/:id/cancel", couponHandler.CancelBatch)

  // Feedback management (Admin only)
  adminProtected.GET("/feedbacks", feedbackHandler.ListFeedbacks)
  adminProtected.GET("/feedbacks/stats", feedbackHandler.GetStats)
  adminProtected.GET("/feedbacks/:id", feedbackHandler.GetFeedback)
  adminProtected.PUT("/feedbacks/:id/status", feedbackHandler.UpdateStatus)
  adminProtected.POST("/feedbacks/:id/reply", feedbackHandler.ReplyFeedback)
  adminProtected.DELETE("/feedbacks/:id", feedbackHandler.DeleteFeedback)
 }

 // Payment callback routes (public, no auth required)
 api.POST("/payment/alipay/notify", alipayHandler.HandleCallback)
 api.POST("/payment/wechat/notify", wechatPayHandler.HandleNotify)

 // WeChat Official Account webhook (public, no auth required)
 wechatOfficialWebhook := api.Group("/wechat-official")
 {
  wechatOfficialWebhook.GET("/webhook", wechatOfficialHandler.HandleVerification)
  wechatOfficialWebhook.POST("/webhook", wechatOfficialHandler.HandleMessage)
 }

 // User routes
```

```
423  userHandler.RegisterRoutes(api)
424
425  // Public subscription routes (no auth required)
426  subscriptionPublic := api.Group("/user/subscription")
427  {
428   subscriptionPublic.GET("/plans", subscriptionHandler.GetPlans)
429   subscriptionPublic.GET("/plans/:code", subscriptionHandler.GetPlanByCode)
430  }
431
432  // Public WeChat routes (no auth required)
433  wechatPublic := api.Group("/user/wechat")
434  {
435   wechatPublic.GET("/login-url", wechatHandler.GetLoginURL)
436   wechatPublic.POST("/callback", userHandler.WeChatCallback)
437  }
438
439  // Public GitHub OAuth routes (no auth required)
440  githubPublic := api.Group("/user/github")
441  {
442   githubPublic.GET("/login-url", userHandler.GetGithubLoginURL)
443   githubPublic.GET("/callback", userHandler.GithubCallback)
444  }
445
446  // User protected routes
447  userProtected := api.Group("/user")
448  userProtected.Use(middleware.UserAuthMiddleware())
449  {
450   userProtected.GET("/profile", userHandler.GetProfile)
451   userProtected.PUT("/profile", userHandler.UpdateProfile)
452   userProtected.POST("/logout", userHandler.Logout)
453   userProtected.POST("/change-password", userHandler.ChangePassword)
454   userProtected.POST("/heartbeat", userHandler.Heartbeat)
455
456   // Account availability check routes (protected)
457   userProtected.POST("/check-email-availability", userHandler.CheckEmailAvailability)
458   userProtected.POST("/check-phone-availability", userHandler.CheckPhoneAvailability)
459
460   // Account binding routes (protected)
461   userProtected.POST("/bind-phone", userHandler.BindPhone)
462   userProtected.POST("/bind-email", userHandler.BindEmail)
463   userProtected.POST("/bind-wechat", userHandler.BindWeChatAccount)
464   userProtected.POST("/bind-github", userHandler.BindGitHubAccount)
465   userProtected.POST("/bind-email-with-password", userHandler.BindEmailWithPassword)
466   userProtected.POST("/bind-phone-with-password", userHandler.BindPhoneWithPassword)
467
468   // Account unbinding routes (protected)
469   userProtected.POST("/unbind-phone", userHandler.UnbindPhone)
470   userProtected.POST("/unbind-email", userHandler.UnbindEmail)
471   userProtected.POST("/unbind-wechat", userHandler.UnbindWeChat)
472   userProtected.POST("/unbind-github", userHandler.UnbindGitHub)
473
474   // Subscription routes (protected)
475   userProtected.GET("/subscription/my", subscriptionHandler.GetMySubscription)
476   userProtected.GET("/subscription/usage", subscriptionHandler.GetMyUsageQuota)
477   userProtected.POST("/subscription/upgrade", subscriptionHandler.UpgradeSubscription)
478   userProtected.POST("/subscription/renew", subscriptionHandler.RenewSubscription)
479   userProtected.GET("/subscription/check-quota", subscriptionHandler.CheckQuota)
```

```
480
481     // Order routes
482     userProtected.POST("/orders/create", orderHandler.CreateOrder)
483     userProtected.GET("/orders/my", orderHandler.GetMyOrders)
484     userProtected.GET("/orders/:order_no", orderHandler.GetOrderDetail)
485     userProtected.POST("/orders/pay", orderHandler.ProcessPayment)
486     userProtected.POST("/orders/cancel", orderHandler.CancelOrder)
487     userProtected.GET("/orders/billing-history", orderHandler.GetBillingHistory)
488
489     // Payment routes (Alipay)
490     userProtected.POST("/payment/alipay/create", alipayHandler.CreatePayment)
491     userProtected.POST("/payment/simulate", alipayHandler.SimulatePayment)
492
493     // Payment routes (WeChat Pay)
494     userProtected.POST("/payment/wechat/create", wechatPayHandler.CreatePayment)
495
496     // Coupon routes (User)
497     userProtected.GET("/coupons", couponHandler.GetMyCoupons)
498     userProtected.GET("/coupons/available", couponHandler.GetAvailableCoupons)
499     userProtected.POST("/coupons/claim", couponHandler.ClaimCoupon)
500     userProtected.POST("/coupons/apply", couponHandler.ApplyCoupon)
501     userProtected.GET("/coupons/calculate-discount", couponHandler.CalculateDiscount)
502
503     // Invitation routes (User)
504     userProtected.POST("/invitation/generate", couponHandler.GenerateInvitation)
505     userProtected.GET("/invitation/my", couponHandler.GetMyInvitation)
506     userProtected.GET("/invitation/statistics", couponHandler.GetInvitationStatistics)
507
508     // Feedback routes (User) - 已登录用户提交反馈会关联用户信息
509     userProtected.POST("/feedback", feedbackHandler.CreateFeedbackWithUser)
510
511     // Conversation routes (legacy)
512     userProtected.GET("/conversations", conversationHandler.GetUserConversations)
513     userProtected.POST("/conversations", conversationHandler.CreateConversation)
514     userProtected.GET("/conversations/:id", conversationHandler.GetConversation)
515     userProtected.DELETE("/conversations/:id", conversationHandler.DeleteConversation)
516     userProtected.POST("/conversations/:id/messages", conversationHandler.AddMessage)
517     userProtected.PUT("/conversations/:id/title", conversationHandler.UpdateConversationTitle)
518     userProtected.GET("/conversations/:id/messages/:messageId/advanced-search", conversationHa
ndler.StreamAdvancedSearch)
519     userProtected.GET("/conversations/:id/messages/:messageId/follow-up-questions", conversati
onHandler.GetFollowUpQuestions)
520     userProtected.POST("/conversations/:id/continue", conversationHandler.StreamContinueConver
sation)
521
522     // Enhanced chat routes (LibreChat-style with message chains)
523     userProtected.POST("/conversations/chat/new", chatHandler.CreateConversationAndChat)
524     userProtected.POST("/conversations/:id/chat/stream", chatHandler.StreamChatMessage)
525     userProtected.POST("/conversations/:id/chat", chatHandler.QuickChat)
526     userProtected.GET("/conversations/:id/tree", chatHandler.GetMessageTree)
527     userProtected.GET("/conversations/:id/history", chatHandler.GetConversationHistory)
528     userProtected.POST("/conversations/messages/:messageId/regenerate", chatHandler.Regenerate
Message)
529     userProtected.POST("/conversations/messages/:messageId/edit", chatHandler.EditMessage)
530     userProtected.GET("/conversations/messages/:messageId/siblings", chatHandler.GetSiblingMes
sages)
531     userProtected.GET("/conversations/messages/:messageId/branch", chatHandler.GetBranchChain)
```

```
532
533     // Peer Review routes (AI审稿人) - 新版任务管理API
534     peerReviewGroup := userProtected.Group("/peer-review")
535     {
536         // 草稿管理（每个用户一个草稿）
537         peerReviewGroup.GET("/draft", peerReviewHandler.GetDraft)      // 获取草稿
538         peerReviewGroup.PUT("/draft", peerReviewHandler.SaveDraft)     // 保存草稿
539         peerReviewGroup.POST("/draft/submit", peerReviewHandler.SubmitDraft) // 提交草稿开始审稿
540
541         // 任务管理
542         peerReviewGroup.POST("/tasks", peerReviewHandler.CreateTask)
543         peerReviewGroup.GET("/tasks", peerReviewHandler.GetTaskList)
544         peerReviewGroup.GET("/tasks/:id", peerReviewHandler.GetTaskDetail)
545         peerReviewGroup.GET("/tasks/:id/progress", peerReviewHandler.GetTaskProgress)
546         peerReviewGroup.GET("/tasks/:id/watch", peerReviewHandler.WatchTaskProgress)
547         peerReviewGroup.DELETE("/tasks/:id", peerReviewHandler.DeleteTask)
548         peerReviewGroup.POST("/tasks/:id/start", peerReviewHandler.StartTask) // 启动草稿任务
549
550         // 文件上传（解析文件并保存到草稿）
551         peerReviewGroup.POST("/upload", peerReviewHandler.UploadFile)
552
553         // 兼容旧接口
554         peerReviewGroup.POST("/run", peerReviewHandler.RunPeerReview)
555         peerReviewGroup.GET("/history", peerReviewHandler.GetReviewHistory)
556     }
557     // 兼容旧接口路由
558     userProtected.GET("/peer-review/:id", peerReviewHandler.GetReviewDetail)
559 }
560
561 // Public API routes (no authentication required)
562 publicAPI := api.Group("/public")
563 {
564     // Public paper detail endpoint - for anonymous access from paper detail page
565     publicAPI.GET("/papers/:paperId", scholarHandler.GetPaper)
566     // Public search endpoint - for anonymous access from home page
567     publicAPI.POST("/papers/search", scholarHandler.SearchPapers)
568     // Public paper batch endpoint - for anonymous access to get multiple papers
569     publicAPI.POST("/papers/batch", scholarHandler.GetPaperBatch)
570     // Public paper autocomplete endpoint - for search suggestions dropdown
571     publicAPI.GET("/papers/autocomplete", scholarHandler.PaperAutocomplete)
572     // Public search suggestions endpoint - combined paper + author suggestions
573     publicAPI.GET("/search/suggestions", scholarHandler.GetSearchSuggestions)
574     // Public paper analysis endpoint - AI-powered abstract analysis
575     publicAPI.POST("/papers/analyze", aiHandler.AnalyzePaper)
576     // Public analysis dimensions endpoint - get system default dimensions
577     publicAPI.GET("/analysis/dimensions", aiHandler.GetSystemDimensions)
578     // Public recommendations endpoint - for anonymous access to similar papers
579     publicAPI.GET("/recommendations/papers/:paperId", scholarHandler.GetRecommendationsForPape
r)
580
581     // Public author endpoints - for anonymous access from author detail page
582     publicAPI.GET("/authors/:authorId", scholarHandler.GetAuthor)
583     publicAPI.GET("/authors/:authorId/papers", scholarHandler.GetAuthorPapers)
584
585     // Public feedback endpoints
586     publicAPI.POST("/feedback", feedbackHandler.CreateFeedback)
587     publicAPI.POST("/feedback/upload", feedbackHandler.UploadAttachments)
```

```
588  publicAPI.GET("/feedback/attachment/:id", feedbackHandler.GetAttachment)
589 }
590
591 // Analysis dimensions routes (protected - requires user authentication)
592 analysisGroup := api.Group("/analysis")
593 analysisGroup.Use(middleware.UserAuthMiddleware())
594 {
595  // Analyze paper with user's custom dimensions
596  analysisGroup.POST("/paper", aiHandler.AnalyzePaper)
597  // Get user's dimensions (custom or system default)
598  analysisGroup.GET("/dimensions", aiHandler.GetAnalysisDimensions)
599  // Create custom dimension
600  analysisGroup.POST("/dimensions", aiHandler.CreateUserDimension)
601  // Update custom dimension
602  analysisGroup.PUT("/dimensions/:id", aiHandler.UpdateUserDimension)
603  // Delete custom dimension
604  analysisGroup.DELETE("/dimensions/:id", aiHandler.DeleteUserDimension)
605  // Reset single dimension to default
606  analysisGroup.DELETE("/dimensions/reset/:dimensionKey", aiHandler.ResetUserDimension)
607  // Reset all dimensions to default
608  analysisGroup.DELETE("/dimensions/reset-all", aiHandler.ResetAllUserDimensions)
609  // Copy system dimensions to user (deprecated, kept for compatibility)
610  analysisGroup.POST("/dimensions/copy-from-system", aiHandler.CopySystemDimensionsToUser)
611 }
612
613 // Semantic Scholar API routes (protected - requires user authentication)
614 scholar := api.Group("/scholar")
615 scholar.Use(middleware.UserAuthMiddleware())
616 {
617  // Paper endpoints
618  scholar.POST("/papers/search", scholarHandler.SearchPapers) // 学术检索（改为POST）
619  scholar.GET("/papers/:paperId", scholarHandler.GetPaper)
620  scholar.POST("/papers/batch", scholarHandler.GetPaperBatch)
621  scholar.GET("/papers/:paperId/citations", scholarHandler.GetPaperCitations)
622  scholar.GET("/papers/:paperId/references", scholarHandler.GetPaperReferences)
623  scholar.GET("/papers/:paperId/authors", scholarHandler.GetPaperAuthors)
624
625  // Author endpoints
626  scholar.GET("/authors/search", scholarHandler.SearchAuthors)
627  scholar.GET("/authors/:authorId", scholarHandler.GetAuthor)
628  scholar.GET("/authors/:authorId/papers", scholarHandler.GetAuthorPapers)
629
630  // Recommendations endpoints
631  scholar.GET("/recommendations/papers/:paperId", scholarHandler.GetRecommendationsForPaper)
632  scholar.POST("/recommendations/papers", scholarHandler.GetRecommendations)
633 }
634
635 // MCP endpoints (protected - requires user authentication)
636 mcpGroup := api.Group("/mcp")
637 mcpGroup.Use(middleware.UserAuthMiddleware())
638 {
639  mcpGroup.GET("/tools", mcpHandler.ListTools)
640  mcpGroup.POST("/call", mcpHandler.CallTool)
641 }
642
643 // Proxy endpoints (protected - requires user authentication)
644 proxyHandler.RegisterRoutes(api, middleware.UserAuthMiddleware())
```

```
645 }
646
647 // Serve uploaded files (feedback attachments)
648 router.Static("/uploads", "./uploads")
649
650 // Health check endpoint
651 router.GET("/health", func(c *gin.Context) {
652  c.JSON(200, gin.H{
653   "status": "ok",
654   "service": "scholar-backend",
655  })
656 })
657
658 // Create context for graceful shutdown
659 ctx, cancel := context.WithCancel(context.Background())
660 defer cancel()
661
662 // Start database connection pool monitoring
663 database.StartConnectionPoolMonitoring(ctx, 5*time.Minute)
664
665 // Start background tasks
666 // Record online user snapshots every 5 minutes
667 go func() {
668  ticker := time.NewTicker(5 * time.Minute)
669  defer ticker.Stop()
670
671  log.Println("Online user snapshot task started")
672
673  // Record initial snapshot
674  if err := userService.RecordOnlineUserSnapshot(); err != nil {
675   log.Printf("Failed to record initial online user snapshot: %v", err)
676  }
677
678  for {
679   select {
680   case <-ticker.C:
681    if err := userService.RecordOnlineUserSnapshot(); err != nil {
682     log.Printf("Failed to record online user snapshot: %v", err)
683    }
684   case <-ctx.Done():
685    log.Println("Online user snapshot task stopped")
686    return
687   }
688  }
689 }()
690
691 // Setup graceful shutdown
692 quit := make(chan os.Signal, 1)
693 signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
694
695 // Start server in a goroutine
696 addr := ":" + cfg.Server.Port
697 log.Printf("Server starting on %s", addr)
698
699 go func() {
700  if err := router.Run(addr); err != nil {
701   log.Fatalf("Failed to start server: %v", err)
```

```
  }
}()

// Wait for interrupt signal
<-quit
log.Println("Shutting down server...")

// Cancel context to stop background tasks
cancel()

// Shutdown cache worker pool gracefully
cache.ShutdownGlobalPool()

// Give background tasks time to cleanup
time.Sleep(2 * time.Second)

log.Println("Server stopped")
}

// feedbackUserRepoAdapter adapts user.Repository to feedback.UserRepository interface
type feedbackUserRepoAdapter struct {
userRepo *user.Repository
}

func (a *feedbackUserRepoAdapter) FindByID(id uint) (*feedback.UserInfo, error) {
u, err := a.userRepo.FindByID(id)
if err != nil || u == nil {
 return nil, err
}
return &feedback.UserInfo{
 ID:       u.ID,
 Username:  u.Username,
 Email:    u.Email,
 Phone:    u.Phone,
 AvatarURL: u.AvatarURL,
}, nil
}

func (a *feedbackUserRepoAdapter) FindByIDs(ids []uint) (map[uint]*feedback.UserInfo, error)
{
users, err := a.userRepo.FindByIDs(ids)
if err != nil {
 return nil, err
}
result := make(map[uint]*feedback.UserInfo)
for id, u := range users {
 result[id] = &feedback.UserInfo{
  ID:       u.ID,
  Username:  u.Username,
  Email:    u.Email,
  Phone:    u.Phone,
  AvatarURL: u.AvatarURL,
 }
}
return result, nil
}
```

```
758 // ========== 文件: backend/internal/admin/dashboard_handler.go ==========
759 // ================================================================================
760 // Scholay学术平台 V1.0
761 // Copyright (c) 2025 成都恩特罐罐文化科技有限公司
762 // 统一社会信用代码: 91510100MAEEHRNE26
763 // All Rights Reserved.
764 // ================================================================================
765
766 package admin
767
768 import (
769  "net/http"
770  "scholar-backend/internal/common/utils"
771  "time"
772
773  "github.com/gin-gonic/gin"
774  "gorm.io/gorm"
775 )
776
777 // DashboardHandler handles dashboard statistics requests
778 type DashboardHandler struct {
779  db *gorm.DB
780 }
781
782 // NewDashboardHandler creates a new dashboard handler
783 func NewDashboardHandler(db *gorm.DB) *DashboardHandler {
784  return &DashboardHandler{db: db}
785 }
786
787 // DashboardStatistics represents overall dashboard statistics
788 type DashboardStatistics struct {
789  // 用户统计
790  TotalUsers      int64            `json:"total_users"`       // 总用户数
791  ActiveUsers30d  int64            `json:"active_users_30d"`  // 30天活跃用户
792  NewUsersToday   int64            `json:"new_users_today"`   // 今日新增
793  NewUsers7d      int64            `json:"new_users_7d"`      // 7天新增
794  NewUsers30d     int64            `json:"new_users_30d"`     // 30天新增
795  OnlineUsers     int64            `json:"online_users"`      // 当前在线用户
796
797  // 订单统计
798  TotalOrders     int64            `json:"total_orders"`      // 总订单数
799  PendingOrders   int64            `json:"pending_orders"`    // 待支付订单
800  PaidOrders      int64            `json:"paid_orders"`       // 已支付订单
801  TotalRevenue    float64          `json:"total_revenue"`     // 总收入
802  RevenueToday    float64          `json:"revenue_today"`     // 今日收入
803  Revenue7d       float64          `json:"revenue_7d"`        // 7天收入
804  Revenue30d      float64          `json:"revenue_30d"`       // 30天收入
805
806  // 订阅统计
807  ActiveSubscriptions int64        `json:"active_subscriptions"` // 活跃订阅数
808  FreeUsers       int64            `json:"free_users"`        // 免费用户
809  PaidUsers       int64            `json:"paid_users"`        // 付费用户
810
811  // 优惠券统计
812  TotalCoupons    int64            `json:"total_coupons"`     // 总优惠券数
813  UsedCoupons     int64            `json:"used_coupons"`      // 已使用
814  UnusedCoupons   int64            `json:"unused_coupons"`    // 未使用
```

```
    ExpiredCoupons     int64          `json:"expired_coupons"`     // 已过期

    // 会话统计
    TotalConversations  int64          `json:"total_conversations"` // 总对话数
    ConversationsToday  int64          `json:"conversations_today"` // 今日对话
    TotalMessages      int64          `json:"total_messages"`      // 总消息数

    // 系统统计
    ThirdPartyAccounts  int64          `json:"third_party_accounts"` // 第三方账号数
    PromptTemplates     int64          `json:"prompt_templates"`    // 提示词模板数

    // 趋势数据 (用于图表)
    UserGrowthChart     []ChartDataPoint `json:"user_growth_chart"`   // 用户增长趋势
    OrderTrendChart     []ChartDataPoint `json:"order_trend_chart"`   // 订单趋势
    RevenueTrendChart   []ChartDataPoint `json:"revenue_trend_chart"` // 收入趋势
}

// ChartDataPoint represents a point in time-series chart
type ChartDataPoint struct {
    Date  string  `json:"date"`  // YYYY-MM-DD format
    Value float64 `json:"value"` // 数值
}

// GetDashboardStatistics handles GET /api/v1/admin/dashboard/statistics
func (h *DashboardHandler) GetDashboardStatistics(c *gin.Context) {
    stats := &DashboardStatistics{}
    now := time.Now()
    today := now.Format("2006-01-02")
    date7dAgo := now.AddDate(0, 0, -7).Format("2006-01-02")
    date30dAgo := now.AddDate(0, 0, -30).Format("2006-01-02")

    // ==========================================
    // 用户统计
    // ==========================================

    // 总用户数
    h.db.Table("users").Count(&stats.TotalUsers)

    // 今日新增用户
    h.db.Table("users").
        Where("DATE(created_at) = ?", today).
        Count(&stats.NewUsersToday)

    // 7天新增用户
    h.db.Table("users").
        Where("DATE(created_at) >= ?", date7dAgo).
        Count(&stats.NewUsers7d)

    // 30天新增用户
    h.db.Table("users").
        Where("DATE(created_at) >= ?", date30dAgo).
        Count(&stats.NewUsers30d)

    // 30天活跃用户 (有登录记录)
    h.db.Table("user_sessions").
        Where("last_active_at >= ?", time.Now().AddDate(0, 0, -30)).
        Distinct("user_id").
```

```
872  Count(&stats.ActiveUsers30d)
873
874  // 当前在线用户 (5分钟内活跃)
875  h.db.Table("user_sessions").
876   Where("last_active_at >= ?", time.Now().Add(-5*time.Minute)).
877   Count(&stats.OnlineUsers)
878
879  // ==========================================
880  // 订单统计
881  // ==========================================
882
883  // 总订单数
884  h.db.Table("orders").Count(&stats.TotalOrders)
885
886  // 待支付订单
887  h.db.Table("orders").
888   Where("status = ?", "pending").
889   Count(&stats.PendingOrders)
890
891  // 已支付订单
892  h.db.Table("orders").
893   Where("status = ?", "paid").
894   Count(&stats.PaidOrders)
895
896  // 总收入
897  h.db.Table("orders").
898   Where("status = ?", "paid").
899   Select("COALESCE(SUM(final_amount), 0)").
900   Scan(&stats.TotalRevenue)
901
902  // 今日收入
903  h.db.Table("orders").
904   Where("status = ? AND DATE(paid_at) = ?", "paid", today).
905   Select("COALESCE(SUM(final_amount), 0)").
906   Scan(&stats.RevenueToday)
907
908  // 7天收入
909  h.db.Table("orders").
910   Where("status = ? AND DATE(paid_at) >= ?", "paid", date7dAgo).
911   Select("COALESCE(SUM(final_amount), 0)").
912   Scan(&stats.Revenue7d)
913
914  // 30天收入
915  h.db.Table("orders").
916   Where("status = ? AND DATE(paid_at) >= ?", "paid", date30dAgo).
917   Select("COALESCE(SUM(final_amount), 0)").
918   Scan(&stats.Revenue30d)
919
920  // ==========================================
921  // 订阅统计
922  // ==========================================
923
924  // 活跃订阅数 (未过期)
925  h.db.Table("user_subscriptions").
926   Where("status = ? AND (expired_at IS NULL OR expired_at > ?)", "active", now).
927   Count(&stats.ActiveSubscriptions)
928
```

```
929 // 免费用户 (订阅计划为免费)
930 h.db.Table("user_subscriptions").
931  Joins("JOIN subscription_plans ON user_subscriptions.plan_id = subscription_plans.id").
932  Where("subscription_plans.price = 0 AND user_subscriptions.status = ?", "active").
933  Count(&stats.FreeUsers)
934
935 // 付费用户
936 h.db.Table("user_subscriptions").
937  Joins("JOIN subscription_plans ON user_subscriptions.plan_id = subscription_plans.id").
938  Where("subscription_plans.price > 0 AND user_subscriptions.status = ?", "active").
939  Count(&stats.PaidUsers)
940
941 // ============================================
942 // 优惠券统计
943 // ============================================
944
945 // 总优惠券数
946 h.db.Table("user_coupons").Count(&stats.TotalCoupons)
947
948 // 已使用优惠券
949 h.db.Table("user_coupons").
950  Where("status = ?", "used").
951  Count(&stats.UsedCoupons)
952
953 // 未使用优惠券
954 h.db.Table("user_coupons").
955  Where("status = ?", "unused").
956  Count(&stats.UnusedCoupons)
957
958 // 已过期优惠券
959 h.db.Table("user_coupons").
960  Where("status = ?", "expired").
961  Count(&stats.ExpiredCoupons)
962
963 // ============================================
964 // 会话统计
965 // ============================================
966
967 // 总对话数
968 h.db.Table("conversations").Count(&stats.TotalConversations)
969
970 // 今日对话数
971 h.db.Table("conversations").
972  Where("DATE(created_at) = ?", today).
973  Count(&stats.ConversationsToday)
974
975 // 总消息数
976 h.db.Table("conversation_messages").Count(&stats.TotalMessages)
977
978 // ============================================
979 // 系统统计
980 // ============================================
981
982 // 第三方账号数
983 h.db.Table("third_party_accounts").Count(&stats.ThirdPartyAccounts)
984
985 // 提示词模板数
```

```
  h.db.Table("prompt_templates").Count(&stats.PromptTemplates)

  // ============================================
  // 趋势图表数据
  // ============================================

  // 用户增长趋势 (最近30天)
  stats.UserGrowthChart = h.getUserGrowthChart(30)

  // 订单趋势 (最近30天)
  stats.OrderTrendChart = h.getOrderTrendChart(30)

  // 收入趋势 (最近30天)
  stats.RevenueTrendChart = h.getRevenueTrendChart(30)

  utils.SuccessResponse(c, http.StatusOK, "Dashboard statistics retrieved successfully", stats
)
}

// getUserGrowthChart 获取用户增长趋势图表数据
func (h *DashboardHandler) getUserGrowthChart(days int) []ChartDataPoint {
  var results []struct {
    Date  string
    Count int64
  }

  h.db.Table("users").
    Select("DATE(created_at) as date, COUNT(*) as count").
    Where("created_at >= ?", time.Now().AddDate(0, 0, -days)).
    Group("DATE(created_at)").
    Order("date ASC").
    Scan(&results)

  chart := make([]ChartDataPoint, 0, len(results))
  for _, r := range results {
    chart = append(chart, ChartDataPoint{
      Date:  r.Date,
      Value: float64(r.Count),
    })
  }

  return chart
}

// getOrderTrendChart 获取订单趋势图表数据
func (h *DashboardHandler) getOrderTrendChart(days int) []ChartDataPoint {
  var results []struct {
    Date  string
    Count int64
  }

  h.db.Table("orders").
    Select("DATE(created_at) as date, COUNT(*) as count").
    Where("created_at >= ? AND status = ?", time.Now().AddDate(0, 0, -days), "paid").
    Group("DATE(created_at)").
    Order("date ASC").
    Scan(&results)
```

```
1042
1043  chart := make([]ChartDataPoint, 0, len(results))
1044  for _, r := range results {
1045   chart = append(chart, ChartDataPoint{
1046    Date:  r.Date,
1047    Value: float64(r.Count),
1048   })
1049  }
1050
1051  return chart
1052 }
1053
1054 // getRevenueTrendChart 获取收入趋势图表数据
1055 func (h *DashboardHandler) getRevenueTrendChart(days int) []ChartDataPoint {
1056  var results []struct {
1057   Date   string
1058   Amount float64
1059  }
1060
1061  h.db.Table("orders").
1062   Select("DATE(paid_at) as date, COALESCE(SUM(final_amount), 0) as amount").
1063   Where("paid_at >= ? AND status = ?", time.Now().AddDate(0, 0, -days), "paid").
1064   Group("DATE(paid_at)").
1065   Order("date ASC").
1066   Scan(&results)
1067
1068  chart := make([]ChartDataPoint, 0, len(results))
1069  for _, r := range results {
1070   chart = append(chart, ChartDataPoint{
1071    Date:  r.Date,
1072    Value: r.Amount,
1073   })
1074  }
1075
1076  return chart
1077 }
1078
1079 // ========== 文件: backend/internal/admin/handler.go ==========
1080 // ============================================================================
1081 // Scholay学术平台 V1.0
1082 // Copyright (c) 2025 成都恩特罐罐文化科技有限公司
1083 // 统一社会信用代码: 91510100MAEEHRNE26
1084 // All Rights Reserved.
1085 // ============================================================================
1086
1087 package admin
1088
1089 import (
1090  "scholar-backend/internal/common/utils"
1091
1092  "github.com/gin-gonic/gin"
1093 )
1094
1095 // Handler handles HTTP requests for admin operations
1096 type Handler struct {
1097  service *Service
1098 }
```

```
1099
1100 // NewHandler creates a new admin handler
1101 func NewHandler(service *Service) *Handler {
1102  return &Handler{service: service}
1103 }
1104
1105 // LoginRequest represents a login request
1106 type LoginRequest struct {
1107  Username string `json:"username" binding:"required"`
1108  Password string `json:"password" binding:"required"`
1109 }
1110
1111 // LoginResponse represents a login response
1112 type LoginResponse struct {
1113  Token string     `json:"token"`
1114  Admin *AdminUser `json:"admin"`
1115 }
1116
1117 // UpdatePasswordRequest represents an update password request
1118 type UpdatePasswordRequest struct {
1119  OldPassword string `json:"old_password" binding:"required"`
1120  NewPassword string `json:"new_password" binding:"required,min=8"`
1121 }
1122
1123 // VerifyPasswordRequest represents a verify password request
1124 type VerifyPasswordRequest struct {
1125  Password string `json:"password" binding:"required"`
1126 }
1127
1128 // Login handles admin login
1129 func (h *Handler) Login(c *gin.Context) {
1130  var req LoginRequest
1131  if err := c.ShouldBindJSON(&req); err != nil {
1132   utils.BadRequest(c, "Invalid request: "+err.Error())
1133   return
1134  }
1135
1136  admin, token, err := h.service.Login(req.Username, req.Password)
1137  if err != nil {
1138   utils.Error(c, 1001, err.Error())
1139   return
1140  }
1141
1142  utils.Success(c, LoginResponse{
1143   Token: token,
1144   Admin: admin,
1145  })
1146 }
1147
1148 // GetProfile handles getting admin profile
1149 func (h *Handler) GetProfile(c *gin.Context) {
1150  userID, exists := c.Get("user_id")
1151  if !exists {
1152   utils.Unauthorized(c, "User not authenticated")
1153   return
1154  }
1155
```

```
1156  admin, err := h.service.GetProfile(userID.(uint))
1157  if err != nil {
1158   utils.Error(c, 1002, "Failed to get profile")
1159   return
1160  }
1161
1162  if admin == nil {
1163   utils.Error(c, 1003, "Admin not found")
1164   return
1165  }
1166
1167  utils.Success(c, admin)
1168 }
1169
1170 // UpdatePassword handles updating admin password
1171 func (h *Handler) UpdatePassword(c *gin.Context) {
1172  userID, exists := c.Get("user_id")
1173  if !exists {
1174   utils.Unauthorized(c, "User not authenticated")
1175   return
1176  }
1177
1178  var req UpdatePasswordRequest
1179  if err := c.ShouldBindJSON(&req); err != nil {
1180   utils.BadRequest(c, "Invalid request: "+err.Error())
1181   return
1182  }
1183
1184  err := h.service.UpdatePassword(userID.(uint), req.OldPassword, req.NewPassword)
1185  if err != nil {
1186   utils.Error(c, 1004, err.Error())
1187   return
1188  }
1189
1190  utils.SuccessWithMessage(c, "Password updated successfully", nil)
1191 }
1192
1193 // VerifyPassword handles verifying admin password (for sensitive operations)
1194 func (h *Handler) VerifyPassword(c *gin.Context) {
1195  userID, exists := c.Get("user_id")
1196  if !exists {
1197   utils.Unauthorized(c, "User not authenticated")
1198   return
1199  }
1200
1201  var req VerifyPasswordRequest
1202  if err := c.ShouldBindJSON(&req); err != nil {
1203   utils.BadRequest(c, "Invalid request: "+err.Error())
1204   return
1205  }
1206
1207  valid, err := h.service.VerifyPassword(userID.(uint), req.Password)
1208  if err != nil {
1209   utils.Error(c, 1005, err.Error())
1210   return
1211  }
1212
```

```
1213  if !valid {
1214   utils.Error(c, 1006, "Invalid password")
1215   return
1216  }
1217
1218  utils.SuccessWithMessage(c, "Password verified successfully", nil)
1219 }
1220
1221 // RegisterRoutes registers admin routes
1222 func (h *Handler) RegisterRoutes(router *gin.RouterGroup) {
1223  admin := router.Group("/admin")
1224  {
1225   admin.POST("/login", h.Login)
1226   // Protected routes (require authentication)
1227   // These will be added in the main server setup with middleware
1228  }
1229 }
1230
1231 // ========== 文件: backend/internal/admin/model.go ==========
1232 // ============================================================================
1233 // Scholay学术平台 V1.0
1234 // Copyright (c) 2025 成都恩特罐罐文化科技有限公司
1235 // 统一社会信用代码: 91510100MAEEHRNE26
1236 // All Rights Reserved.
1237 // ============================================================================
1238
1239 package admin
1240
1241 import (
1242  "time"
1243 )
1244
1245 // AdminUser represents an admin user in the system
1246 type AdminUser struct {
1247  ID        uint      `gorm:"primaryKey" json:"id"`
1248  Username    string    `gorm:"unique;not null;size:50" json:"username"`
1249  PasswordHash string    `gorm:"not null;size:255" json:"-"`
1250  Email      *string   `gorm:"size:100" json:"email"`
1251  Status     int8      `gorm:"not null;default:1" json:"status"` // 1=active, 0=inactive
1252  LastLoginAt *time.Time `json:"last_login_at"`
1253  CreatedAt   time.Time  `json:"created_at"`
1254  UpdatedAt   time.Time  `json:"updated_at"`
1255 }
1256
1257 // TableName specifies the table name for AdminUser
1258 func (AdminUser) TableName() string {
1259  return "admin_users"
1260 }
1261
1262 // ========== 文件: backend/internal/admin/repository.go ==========
1263 // ============================================================================
1264 // Scholay学术平台 V1.0
1265 // Copyright (c) 2025 成都恩特罐罐文化科技有限公司
1266 // 统一社会信用代码: 91510100MAEEHRNE26
1267 // All Rights Reserved.
1268 // ============================================================================
1269
```

```
1270 package admin
1271
1272 import (
1273  "errors"
1274  "time"
1275
1276  "gorm.io/gorm"
1277 )
1278
1279 // Repository handles database operations for admin users
1280 type Repository struct {
1281  db *gorm.DB
1282 }
1283
1284 // NewRepository creates a new admin repository
1285 func NewRepository(db *gorm.DB) *Repository {
1286  return &Repository{db: db}
1287 }
1288
1289 // FindByUsername finds an admin user by username
1290 func (r *Repository) FindByUsername(username string) (*AdminUser, error) {
1291  var admin AdminUser
1292  err := r.db.Where("username = ?", username).First(&admin).Error
1293  if err != nil {
1294   if errors.Is(err, gorm.ErrRecordNotFound) {
1295    return nil, nil
1296   }
1297   return nil, err
1298  }
1299  return &admin, nil
1300 }
1301
1302 // FindByID finds an admin user by ID
1303 func (r *Repository) FindByID(id uint) (*AdminUser, error) {
1304  var admin AdminUser
1305  err := r.db.First(&admin, id).Error
1306  if err != nil {
1307   if errors.Is(err, gorm.ErrRecordNotFound) {
1308    return nil, nil
1309   }
1310   return nil, err
1311  }
1312  return &admin, nil
1313 }
1314
1315 // Create creates a new admin user
1316 func (r *Repository) Create(admin *AdminUser) error {
1317  return r.db.Create(admin).Error
1318 }
1319
1320 // Update updates an admin user
1321 func (r *Repository) Update(admin *AdminUser) error {
1322  return r.db.Save(admin).Error
1323 }
1324
1325 // UpdatePassword updates an admin user's password
1326 func (r *Repository) UpdatePassword(id uint, passwordHash string) error {
```

```
 1327  return r.db.Model(&AdminUser{}).Where("id = ?", id).Update("password_hash", passwordHash).Er
ror
 1328 }
 1329
 1330 // UpdateLastLogin updates the last login timestamp
 1331 func (r *Repository) UpdateLastLogin(id uint) error {
 1332  now := time.Now()
 1333  return r.db.Model(&AdminUser{}).Where("id = ?", id).Update("last_login_at", now).Error
 1334 }
 1335
 1336 // ========== 文件: backend/internal/admin/service.go ==========
 1337 // ================================================================================
 1338 // Scholay学术平台 V1.0
 1339 // Copyright (c) 2025 成都恩特罐罐文化科技有限公司
 1340 // 统一社会信用代码: 91510100MAEEHRNE26
 1341 // All Rights Reserved.
 1342 // ================================================================================
 1343
 1344 package admin
 1345
 1346 import (
 1347  "errors"
 1348  "scholar-backend/internal/common/utils"
 1349 )
 1350
 1351 // Service handles business logic for admin operations
 1352 type Service struct {
 1353  repo *Repository
 1354 }
 1355
 1356 // NewService creates a new admin service
 1357 func NewService(repo *Repository) *Service {
 1358  return &Service{repo: repo}
 1359 }
 1360
 1361 // Login authenticates an admin user
 1362 func (s *Service) Login(username, password string) (*AdminUser, string, error) {
 1363  admin, err := s.repo.FindByUsername(username)
 1364  if err != nil {
 1365   return nil, "", err
 1366  }
 1367
 1368  if admin == nil {
 1369   return nil, "", errors.New("invalid username or password")
 1370  }
 1371
 1372  if admin.Status != 1 {
 1373   return nil, "", errors.New("account is inactive")
 1374  }
 1375
 1376  if !utils.CheckPassword(password, admin.PasswordHash) {
 1377   return nil, "", errors.New("invalid username or password")
 1378  }
 1379
 1380  // Generate JWT token
 1381  token, err := utils.GenerateToken(admin.ID, admin.Username, "admin")
 1382  if err != nil {
```

```
1383  return nil, "", err
1384 }
1385
1386 // Update last login time
1387 _ = s.repo.UpdateLastLogin(admin.ID)
1388
1389 return admin, token, nil
1390 }
1391
1392 // UpdatePassword updates an admin user's password
1393 func (s *Service) UpdatePassword(id uint, oldPassword, newPassword string) error {
1394  admin, err := s.repo.FindByID(id)
1395  if err != nil {
1396   return err
1397  }
1398
1399  if admin == nil {
1400   return errors.New("admin user not found")
1401  }
1402
1403  if !utils.CheckPassword(oldPassword, admin.PasswordHash) {
1404   return errors.New("invalid old password")
1405  }
1406
1407  hashedPassword, err := utils.HashPassword(newPassword)
1408  if err != nil {
1409   return err
1410  }
1411
1412  return s.repo.UpdatePassword(id, hashedPassword)
1413 }
1414
1415 // VerifyPassword verifies an admin user's password (for sensitive operations)
1416 func (s *Service) VerifyPassword(id uint, password string) (bool, error) {
1417  admin, err := s.repo.FindByID(id)
1418  if err != nil {
1419   return false, err
1420  }
1421
1422  if admin == nil {
1423   return false, errors.New("admin user not found")
1424  }
1425
1426  return utils.CheckPassword(password, admin.PasswordHash), nil
1427 }
1428
1429 // GetProfile gets an admin user's profile
1430 func (s *Service) GetProfile(id uint) (*AdminUser, error) {
1431  return s.repo.FindByID(id)
1432 }
1433
1434 // InitializeDefaultAdmin creates the default admin user if not exists
1435 func (s *Service) InitializeDefaultAdmin(username, password string) error {
1436  existing, err := s.repo.FindByUsername(username)
1437  if err != nil {
1438   return err
1439  }
```

```
1440
1441  if existing != nil {
1442   // Default admin already exists
1443   return nil
1444  }
1445
1446  hashedPassword, err := utils.HashPassword(password)
1447  if err != nil {
1448   return err
1449  }
1450
1451  admin := &AdminUser{
1452   Username:    username,
1453   PasswordHash: hashedPassword,
1454   Status:      1,
1455  }
1456
1457  return s.repo.Create(admin)
1458 }
1459
1460 // ========== 文件: backend/internal/ai/handler.go ==========
1461 // ============================================================================
1462 // Scholay学术平台 V1.0
1463 // Copyright (c) 2025 成都恩特罐罐文化科技有限公司
1464 // 统一社会信用代码: 91510100MAEEHRNE26
1465 // All Rights Reserved.
1466 // ============================================================================
1467
1468 package ai
1469
1470 import (
1471  "scholar-backend/internal/common/utils"
1472  "strconv"
1473
1474  "github.com/gin-gonic/gin"
1475 )
1476
1477 type Handler struct {
1478  service *Service
1479 }
1480
1481 func NewHandler(service *Service) *Handler {
1482  return &Handler{
1483   service: service,
1484  }
1485 }
1486
1487 // ListPromptTemplates lists all prompt templates
1488 func (h *Handler) ListPromptTemplates(c *gin.Context) {
1489  templates, err := h.service.ListPromptTemplates()
1490  if err != nil {
1491   utils.Error(c, 500, "Failed to list templates: "+err.Error())
1492   return
1493  }
1494
1495  utils.Success(c, gin.H{
1496   "templates": templates,
```

```
 1497   "total":    len(templates),
 1498  })
 1499 }
 1500
```

==========

第二部分：后30页源代码

==========

```
125980  .settings-back-btn svg {
125981    width: 18px;
125982    height: 18px;
125983  }
125984
125985  .settings-title {
125986    font-size: 24px;
125987  }
125988
125989  .settings-section h2 {
125990    font-size: 20px;
125991  }
125992
125993  .avatar-circle {
125994    width: 100px;
125995    height: 100px;
125996    font-size: 40px;
125997  }
125998
125999  .form-group input,
126000  .form-group textarea {
126001    padding: 10px 14px;
126002    font-size: 16px; /* Prevent zoom on iOS */
126003  }
126004
126005  .feature-grid {
126006    grid-template-columns: 1fr;
126007  }
126008
126009  .subscription-plans {
126010    grid-template-columns: 1fr;
126011  }
126012
126013  .subscription-header {
126014    flex-direction: column;
126015    gap: 16px;
126016  }
126017
126018  .btn-renew {
126019    width: 100%;
126020  }
126021 }
126022
126023 /* ============================================
126024    日间模式样式 - Subscription
126025    ============================================ */
126026 [data-theme="light"] .subscription-loading {
126027   color: #6b7280;
```

```
126028 }
126029
126030 [data-theme="light"] .subscription-loading .spinner {
126031   border: 3px solid rgba(0, 0, 0, 0.1);
126032   border-top-color: #10a37f;
126033 }
126034
126035 [data-theme="light"] .subscription-card {
126036   background: rgba(255, 255, 255, 0.95);
126037   border: 1px solid rgba(0, 0, 0, 0.08);
126038   box-shadow: 0 4px 20px rgba(0, 0, 0, 0.06);
126039 }
126040
126041 [data-theme="light"] .subscription-card:hover {
126042   border-color: rgba(0, 0, 0, 0.12);
126043 }
126044
126045 [data-theme="light"] .subscription-card-header {
126046   background: rgba(0, 0, 0, 0.02);
126047   border-bottom: 1px solid rgba(0, 0, 0, 0.06);
126048 }
126049
126050 [data-theme="light"] .subscription-expiry {
126051   color: #6b7280;
126052 }
126053
126054 [data-theme="light"] .subscription-desc {
126055   color: #4b5563;
126056 }
126057
126058 [data-theme="light"] .subscription-compact {
126059   background: linear-gradient(135deg,
126060     rgba(255, 255, 255, 0.95) 0%,
126061     rgba(248, 250, 252, 0.98) 100%
126062   );
126063   border: 1px solid rgba(0, 0, 0, 0.08);
126064   box-shadow: 0 4px 20px rgba(0, 0, 0, 0.06);
126065 }
126066
126067 [data-theme="light"] .subscription-compact:hover {
126068   border-color: rgba(0, 0, 0, 0.12);
126069 }
126070
126071 [data-theme="light"] .subscription-compact-expiry {
126072   color: #6b7280;
126073 }
126074
126075 [data-theme="light"] .subscription-compact-hint {
126076   color: #9ca3af;
126077 }
126078
126079 [data-theme="light"] .btn-upgrade-compact {
126080   background: rgba(0, 0, 0, 0.04);
126081   border: 1px solid rgba(0, 0, 0, 0.1);
126082   color: #374151;
126083 }
126084
```

```
126085 [data-theme="light"] .btn-upgrade-compact:hover {
126086   background: linear-gradient(135deg, rgba(16, 163, 127, 0.12) 0%, rgba(16, 163, 127, 0.06) 1
00%);
126087   border-color: rgba(16, 163, 127, 0.25);
126088   color: #10a37f;
126089 }
126090
126091 [data-theme="light"] .skeleton-badge,
126092 [data-theme="light"] .skeleton-text,
126093 [data-theme="light"] .skeleton-btn {
126094   background: linear-gradient(90deg, rgba(0, 0, 0, 0.04) 25%, rgba(0, 0, 0, 0.08) 50%, rgba(0
, 0, 0, 0.04) 75%);
126095   background-size: 200% 100%;
126096 }
126097
126098 [data-theme="light"] .billing-section-header h3 {
126099   color: #1f2937;
126100 }
126101
126102 [data-theme="light"] .billing-list {
126103   background: linear-gradient(135deg,
126104     rgba(255, 255, 255, 0.95) 0%,
126105     rgba(248, 250, 252, 0.98) 100%
126106   );
126107   border: 1px solid rgba(0, 0, 0, 0.08);
126108 }
126109
126110 [data-theme="light"] .billing-item {
126111   border-bottom: 1px solid rgba(0, 0, 0, 0.06);
126112 }
126113
126114 [data-theme="light"] .billing-item:hover {
126115   background: rgba(0, 0, 0, 0.02);
126116 }
126117
126118 [data-theme="light"] .billing-plan-name {
126119   color: #1f2937;
126120 }
126121
126122 [data-theme="light"] .billing-cycle-tag {
126123   background: rgba(0, 0, 0, 0.06);
126124   color: #6b7280;
126125 }
126126
126127 [data-theme="light"] .billing-original {
126128   color: #9ca3af;
126129 }
126130
126131 [data-theme="light"] .billing-order-no {
126132   color: #9ca3af;
126133 }
126134
126135 [data-theme="light"] .billing-time {
126136   color: #6b7280;
126137 }
126138
126139 [data-theme="light"] .current-subscription-card {
```

```
126140   background: rgba(255, 255, 255, 0.95);
126141   border: 1px solid rgba(0, 0, 0, 0.08);
126142 }
126143
126144 [data-theme="light"] .current-subscription-card:hover {
126145   border-color: rgba(0, 0, 0, 0.12);
126146 }
126147
126148 [data-theme="light"] .subscription-header h3 {
126149   color: #1f2937;
126150 }
126151
126152 [data-theme="light"] .expiry-date {
126153   color: #6b7280;
126154 }
126155
126156 [data-theme="light"] .subscription-description {
126157   background: rgba(0, 0, 0, 0.03);
126158 }
126159
126160 [data-theme="light"] .subscription-description p {
126161   color: #4b5563;
126162 }
126163
126164 [data-theme="light"] .billing-cycle-toggle {
126165   background: rgba(0, 0, 0, 0.04);
126166   border: 1px solid rgba(0, 0, 0, 0.08);
126167 }
126168
126169 [data-theme="light"] .billing-cycle-toggle button {
126170   color: #6b7280;
126171 }
126172
126173 [data-theme="light"] .subscription-plan-card {
126174   background: rgba(255, 255, 255, 0.95);
126175   border: 1px solid rgba(0, 0, 0, 0.08);
126176 }
126177
126178 [data-theme="light"] .subscription-plan-card:hover {
126179   border-color: rgba(0, 0, 0, 0.15);
126180 }
126181
126182 [data-theme="light"] .subscription-plan-card .plan-header {
126183   border-bottom: 1px solid rgba(0, 0, 0, 0.08);
126184 }
126185
126186 [data-theme="light"] .subscription-plan-card .plan-header h4 {
126187   color: #1f2937;
126188 }
126189
126190 [data-theme="light"] .price-currency {
126191   color: #4b5563;
126192 }
126193
126194 [data-theme="light"] .price-amount {
126195   color: #1f2937;
126196 }
```

```
126197
126198 [data-theme="light"] .price-period {
126199   color: #6b7280;
126200 }
126201
126202 [data-theme="light"] .total-price {
126203   color: #9ca3af;
126204 }
126205
126206 [data-theme="light"] .plan-feature {
126207   color: #4b5563;
126208 }
126209
126210 [data-theme="light"] .subscription-link {
126211   background: rgba(0, 0, 0, 0.03);
126212   border: 1px solid rgba(0, 0, 0, 0.08);
126213 }
126214
126215 [data-theme="light"] .subscription-link p {
126216   color: #6b7280;
126217 }
126218
126219 [data-theme="light"] .feature-col h4 {
126220   color: #374151;
126221 }
126222
126223 [data-theme="light"] .feature-item-small {
126224   background: rgba(0, 0, 0, 0.03);
126225 }
126226
126227 [data-theme="light"] .feature-item-small .feature-label {
126228   color: #6b7280;
126229 }
126230
126231 [data-theme="light"] .feature-item-small .feature-value {
126232   color: #374151;
126233 }
126234
126235 [data-theme="light"] .upgrade-section h3 {
126236   color: #1f2937;
126237 }
126238
126239 [data-theme="light"] .upgrade-desc {
126240   color: #6b7280;
126241 }
126242
126243
126244 -- ========== 文件: docker/mysql/init/01_schema.sql ==========
126245 -- ================================================================================
126246 -- Scholay学术平台 V1.0
126247 -- Copyright (c) 2025 成都恩特罐罐文化科技有限公司
126248 -- 统一社会信用代码: 91510100MAEEHRNE26
126249 -- All Rights Reserved.
126250 -- ================================================================================
126251
126252 -- ==========================================
126253 -- Scholar Database Schema
```

```
126254 -- 此文件包含所有表结构、索引定义
126255 -- 注意: 仅包含DDL语句，不包含数据
126256 -- ============================================
126257
126258 SET NAMES utf8mb4;
126259 SET CHARACTER SET utf8mb4;
126260 SET foreign_key_checks = 0;
126261
126262 -- ============================================
126263 -- 核心用户和认证表
126264 -- ============================================
126265
126266 -- 管理员用户表
126267 CREATE TABLE IF NOT EXISTS `admin_users` (
126268   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126269   `username` VARCHAR(50) NOT NULL,
126270   `password_hash` VARCHAR(255) NOT NULL,
126271   `email` VARCHAR(100),
126272   `status` TINYINT NOT NULL DEFAULT 1 COMMENT '1=active, 0=inactive',
126273   `last_login_at` DATETIME,
126274   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126275   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126276   PRIMARY KEY (`id`),
126277   UNIQUE INDEX `uk_username` (`username`),
126278   INDEX `idx_status` (`status`)
126279 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='管理员用户表';
126280
126281 -- 用户表（支持多种认证方式）
126282 CREATE TABLE IF NOT EXISTS `users` (
126283   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126284   `username` VARCHAR(50),
126285   `email` VARCHAR(100),
126286   `phone` VARCHAR(20),
126287   `password_hash` VARCHAR(255),
126288   `avatar_url` VARCHAR(255),
126289   `bio` TEXT COMMENT '用户简介',
126290   `location` VARCHAR(100) COMMENT '用户位置',
126291   `website` VARCHAR(255) COMMENT '个人网站',
126292   `status` TINYINT NOT NULL DEFAULT 1 COMMENT '1=active, 0=inactive, 2=suspended',
126293   `email_verified` BOOLEAN NOT NULL DEFAULT FALSE,
126294   `phone_verified` BOOLEAN NOT NULL DEFAULT FALSE,
126295   `subscription_plan` VARCHAR(20) NOT NULL DEFAULT 'free' COMMENT 'free, pro, max',
126296   `subscription_expires_at` DATETIME COMMENT '订阅过期时间（免费用户为NULL）',
126297   `last_login_at` DATETIME,
126298   `last_login_ip` VARCHAR(45) COMMENT '最后登录IP',
126299   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126300   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126301   PRIMARY KEY (`id`),
126302   UNIQUE INDEX `uk_username` (`username`),
126303   UNIQUE INDEX `uk_email` (`email`),
126304   UNIQUE INDEX `uk_phone` (`phone`),
126305   INDEX `idx_status` (`status`),
126306   INDEX `idx_subscription_plan` (`subscription_plan`),
126307   INDEX `idx_location` (`location`),
126308   INDEX `idx_last_login_at` (`last_login_at`)
126309 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户表';
126310
```

```
-- 用户认证提供者（OAuth、微信等）
CREATE TABLE IF NOT EXISTS `user_auth_providers` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `user_id` BIGINT UNSIGNED NOT NULL,
  `provider` VARCHAR(20) NOT NULL COMMENT 'email, phone, wechat, google等',
  `provider_user_id` VARCHAR(255) NOT NULL COMMENT '第三方平台用户ID（微信为openID）',
  `provider_union_id` VARCHAR(255) COMMENT '第三方平台联合ID（微信UnionID，用于跨应用识别同一用户）',
  `provider_data` JSON COMMENT '第三方平台额外数据',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_provider_user` (`provider`, `provider_user_id`),
  UNIQUE KEY `uk_provider_union` (`provider`, `provider_union_id`),
  INDEX `idx_user_id` (`user_id`),
  INDEX `idx_union_id` (`provider_union_id`),
  CONSTRAINT `fk_user_auth_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE
CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户认证提供者';

-- ============================================
-- 会话和统计表
-- ============================================

-- 用户会话（在线状态跟踪）
CREATE TABLE IF NOT EXISTS `user_sessions` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `user_id` BIGINT UNSIGNED NOT NULL,
  `token` VARCHAR(500) NOT NULL,
  `ip` VARCHAR(45),
  `user_agent` VARCHAR(500),
  `last_activity` DATETIME NOT NULL,
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `expires_at` DATETIME NOT NULL,
  PRIMARY KEY (`id`),
  INDEX `idx_user_id` (`user_id`),
  INDEX `idx_token` (`token`(64)),  -- Optimized: index only first 64 chars to reduce storage
 overhead
  -- Removed redundant single-column indexes that are covered by composite indexes
  INDEX `idx_active_sessions` (`last_activity`, `expires_at`),
  INDEX `idx_user_activity` (`user_id`, `last_activity` DESC),
  FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户会话';

-- 用户统计（每日汇总数据）
CREATE TABLE IF NOT EXISTS `user_statistics` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `date` DATE NOT NULL UNIQUE COMMENT '统计日期',
  `new_users` INT NOT NULL DEFAULT 0 COMMENT '当日新增用户',
  `total_users` INT NOT NULL DEFAULT 0 COMMENT '累计用户总数',
  `peak_online_users` INT NOT NULL DEFAULT 0 COMMENT '当日最高在线用户数',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE INDEX `uk_date` (`date`),
  INDEX `idx_date` (`date`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='每日用户统计';
```

```
126366 -- 在线用户快照（实时图表数据）
126367 CREATE TABLE IF NOT EXISTS `online_user_snapshots` (
126368   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126369   `timestamp` DATETIME NOT NULL COMMENT '快照时间',
126370   `online_count` INT NOT NULL DEFAULT 0 COMMENT '在线用户数',
126371   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126372   PRIMARY KEY (`id`),
126373   INDEX `idx_timestamp` (`timestamp`)
126374 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='在线用户快照';
126375
126376 -- ============================================
126377 -- 对话和消息表
126378 -- ============================================
126379
126380 -- 对话表
126381 CREATE TABLE IF NOT EXISTS `conversations` (
126382   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126383   `user_id` BIGINT UNSIGNED NOT NULL,
126384   `title` VARCHAR(255) NOT NULL COMMENT '对话标题（从首条消息生成）',
126385   `conversation_type` VARCHAR(20) NOT NULL DEFAULT 'search' COMMENT 'chat或search',
126386   `model_level` VARCHAR(20) DEFAULT 'low' COMMENT '模型级别: low, mid, high',
126387   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126388   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126389   PRIMARY KEY (`id`),
126390   INDEX `idx_user_id` (`user_id`),
126391   INDEX `idx_updated_at` (`updated_at`),
126392   INDEX `idx_conversation_type` (`conversation_type`),
126393   INDEX `idx_user_type` (`user_id`, `conversation_type`),
126394   INDEX `idx_user_updated` (`user_id`, `updated_at` DESC),
126395   INDEX `idx_user_type_updated` (`user_id`, `conversation_type`, `updated_at` DESC),
126396   FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE
126397 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户对话历史';
126398
126399 -- 对话消息表（支持消息链）
126400 CREATE TABLE IF NOT EXISTS `conversation_messages` (
126401   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126402   `message_id` VARCHAR(36) NOT NULL COMMENT '消息UUID',
126403   `conversation_id` BIGINT UNSIGNED NOT NULL,
126404   `parent_message_id` VARCHAR(36) COMMENT '父消息ID（支持分支）',
126405   `role` VARCHAR(20) NOT NULL COMMENT 'user, assistant, system',
126406   `content` TEXT NOT NULL COMMENT '消息内容',
126407   `search_query` VARCHAR(500) COMMENT '搜索查询（如果是搜索消息）',
126408   `search_results` JSON COMMENT 'Semantic Scholar搜索结果',
126409   `search_results_total` INT NOT NULL DEFAULT 0 COMMENT '搜索结果总数',
126410   `follow_up_questions` TEXT COMMENT '延伸思考问题（仅最新一轮assistant消息保存）',
126411   `token_count` INT DEFAULT 0 COMMENT 'Token计数',
126412   `model` VARCHAR(50) COMMENT 'AI模型',
126413   `finish_reason` VARCHAR(20) COMMENT '完成原因',
126414   `error` BOOLEAN DEFAULT FALSE COMMENT '是否出错',
126415   `unfinished` BOOLEAN DEFAULT FALSE COMMENT '是否未完成',
126416   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126417   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126418   PRIMARY KEY (`id`),
126419   UNIQUE INDEX `uk_message_id` (`message_id`),
126420   INDEX `idx_conversation_id` (`conversation_id`),
126421   INDEX `idx_parent_message_id` (`parent_message_id`),
126422   INDEX `idx_created_at` (`created_at`),
```

```
126423   INDEX `idx_conversation_created` (`conversation_id`, `created_at`),
126424   INDEX `idx_message_chain` (`conversation_id`, `parent_message_id`, `created_at`),
126425   INDEX `idx_parent` (`parent_message_id`, `conversation_id`),
126426   FOREIGN KEY (`conversation_id`) REFERENCES `conversations`(`id`) ON DELETE CASCADE
126427 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='对话消息';
126428
126429 -- ============================================
126430 -- 订阅和配额表
126431 -- ============================================
126432
126433 -- 订阅计划表
126434 CREATE TABLE IF NOT EXISTS `subscription_plans` (
126435   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126436   `plan_code` VARCHAR(20) NOT NULL COMMENT '计划代码: free, pro, max',
126437   `plan_name` VARCHAR(50) NOT NULL COMMENT '计划名称',
126438   `price_monthly` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '月费原价',
126439   `current_price_monthly` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '月费实时价格',
126440   `price_yearly` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '年费原价',
126441   `current_price_yearly` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '年费实时价格',
126442   `yearly_discount_enabled` TINYINT NOT NULL DEFAULT 0 COMMENT '年费是否使用折扣计算: 1=使用折扣, 0=直接定价',
126443   `yearly_discount_percent` DECIMAL(5,2) NOT NULL DEFAULT 0.00 COMMENT '年费折扣百分比(0-100)',
126444   `features` JSON NOT NULL COMMENT '计划功能和配额',
126445   `is_active` TINYINT NOT NULL DEFAULT 1 COMMENT '1=启用, 0=停用',
126446   `sort_order` INT NOT NULL DEFAULT 0 COMMENT '显示顺序',
126447   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126448   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126449   PRIMARY KEY (`id`),
126450   UNIQUE INDEX `uk_plan_code` (`plan_code`),
126451   INDEX `idx_is_active` (`is_active`),
126452   INDEX `idx_sort_order` (`sort_order`),
126453   INDEX `idx_active_type` (`is_active`, `plan_code`)
126454 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='订阅计划';
126455
126456 -- 用户订阅表
126457 CREATE TABLE IF NOT EXISTS `user_subscriptions` (
126458   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126459   `user_id` BIGINT UNSIGNED NOT NULL,
126460   `plan_code` VARCHAR(20) NOT NULL COMMENT '计划代码',
126461   `start_date` DATETIME NOT NULL COMMENT '开始日期',
126462   `end_date` DATETIME COMMENT '结束日期',
126463   `status` VARCHAR(20) NOT NULL DEFAULT 'active' COMMENT 'active, expired, cancelled',
126464   `auto_renew` TINYINT NOT NULL DEFAULT 0 COMMENT '1=自动续费',
126465   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126466   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126467   PRIMARY KEY (`id`),
126468   INDEX `idx_user_id` (`user_id`),
126469   INDEX `idx_status` (`status`),
126470   INDEX `idx_end_date` (`end_date`),
126471   INDEX `idx_user_status` (`user_id`, `status`),
126472   INDEX `idx_user_active` (`user_id`, `status`, `end_date`),
126473   FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE
126474 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户订阅记录';
126475
126476 -- 使用配额表
126477 CREATE TABLE IF NOT EXISTS `usage_quotas` (
126478   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126479   `user_id` BIGINT UNSIGNED NOT NULL,
```

```
126480   `quota_date` DATE NOT NULL COMMENT '使用日期',
126481   `normal_search_count` INT NOT NULL DEFAULT 0 COMMENT '普通搜索次数',
126482   `advanced_search_low_count` INT NOT NULL DEFAULT 0 COMMENT '高级搜索(低)次数',
126483   `advanced_search_mid_count` INT NOT NULL DEFAULT 0 COMMENT '高级搜索(中)次数',
126484   `advanced_search_high_count` INT NOT NULL DEFAULT 0 COMMENT '高级搜索(高)次数',
126485   `ai_chat_normal_count` INT NOT NULL DEFAULT 0 COMMENT 'AI对话(普通)次数',
126486   `ai_chat_mid_count` INT NOT NULL DEFAULT 0 COMMENT 'AI对话(中)次数',
126487   `ai_chat_high_count` INT NOT NULL DEFAULT 0 COMMENT 'AI对话(高)次数',
126488   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126489   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126490   PRIMARY KEY (`id`),
126491   UNIQUE KEY `uk_user_date` (`user_id`, `quota_date`),
126492   INDEX `idx_user_id` (`user_id`),
126493   INDEX `idx_quota_date` (`quota_date`),
126494   INDEX `idx_user_date_desc` (`user_id`, `quota_date` DESC),
126495   FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE
126496 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='每日使用配额跟踪';
126497
126498 -- ============================================
126499 -- 订单和支付表
126500 -- ============================================
126501
126502 -- 订单表
126503 CREATE TABLE IF NOT EXISTS `orders` (
126504   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126505   `user_id` BIGINT UNSIGNED NOT NULL,
126506   `order_no` VARCHAR(64) NOT NULL COMMENT '订单号',
126507   `plan_code` VARCHAR(20) NOT NULL COMMENT '订阅计划代码',
126508   `plan_name` VARCHAR(50) NOT NULL COMMENT '下单时的计划名称',
126509   `billing_cycle` VARCHAR(20) NOT NULL COMMENT 'monthly或yearly',
126510   `amount` DECIMAL(10,2) NOT NULL COMMENT '订单原价（折扣前）',
126511   `coupon_id` BIGINT UNSIGNED COMMENT '使用的折扣券ID',
126512   `discount_amount` DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '折扣金额',
126513   `final_amount` DECIMAL(10,2) NOT NULL COMMENT '最终支付金额（折扣后）',
126514   `status` VARCHAR(20) NOT NULL DEFAULT 'pending' COMMENT 'pending, paid, cancelled, refunded
',
126515   `payment_method` VARCHAR(50) COMMENT '支付方式',
126516   `payment_time` DATETIME COMMENT '支付完成时间',
126517   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126518   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126519   PRIMARY KEY (`id`),
126520   UNIQUE INDEX `uk_order_no` (`order_no`),
126521   INDEX `idx_user_id` (`user_id`),
126522   INDEX `idx_coupon_id` (`coupon_id`),
126523   INDEX `idx_status` (`status`),
126524   INDEX `idx_created_at` (`created_at`),
126525   INDEX `idx_user_created` (`user_id`, `created_at` DESC),
126526   INDEX `idx_status_created` (`status`, `created_at` DESC),
126527   CONSTRAINT `fk_orders_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CAS
CADE
126528 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='订阅订单';
126529
126530 -- 支付交易表
126531 CREATE TABLE IF NOT EXISTS `payment_transactions` (
126532   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126533   `order_id` BIGINT UNSIGNED NOT NULL,
126534   `transaction_no` VARCHAR(100) COMMENT '外部交易号',
```

```
126535   `payment_method` VARCHAR(50) NOT NULL COMMENT '支付方式',
126536   `amount` DECIMAL(10,2) NOT NULL,
126537   `status` VARCHAR(20) NOT NULL DEFAULT 'pending' COMMENT 'pending, success, failed',
126538   `payment_data` JSON COMMENT '支付额外数据',
126539   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126540   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126541   PRIMARY KEY (`id`),
126542   INDEX `idx_order_id` (`order_id`),
126543   INDEX `idx_transaction_no` (`transaction_no`),
126544   CONSTRAINT `fk_payment_order` FOREIGN KEY (`order_id`) REFERENCES `orders` (`id`) ON DELETE
 CASCADE
126545 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='支付交易记录';
126546
126547 -- ============================================
126548 -- 折扣券系统表
126549 -- ============================================
126550
126551 -- 折扣券模板表
126552 CREATE TABLE IF NOT EXISTS `coupon_templates` (
126553   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126554   `code` VARCHAR(50) NOT NULL COMMENT '折扣券代码',
126555   `name` VARCHAR(100) NOT NULL COMMENT '折扣券名称',
126556   `description` TEXT COMMENT '折扣券描述',
126557   `type` ENUM('percentage', 'fixed', 'price_off') NOT NULL DEFAULT 'percentage' COMMENT '折扣类型
',
126558   `discount_value` DECIMAL(10,2) NOT NULL COMMENT '折扣值',
126559
126560   -- 使用条件
126561   `min_purchase_amount` DECIMAL(10,2) DEFAULT 0 COMMENT '最低消费金额',
126562   `applicable_plans` JSON COMMENT '适用的订阅计划ID列表',
126563   `applicable_products` JSON COMMENT '适用的产品类型列表',
126564
126565   -- 发放规则
126566   `distribution_type` ENUM('manual', 'auto_invite', 'auto_register', 'batch', 'targeted') NOT
 NULL DEFAULT 'manual' COMMENT '发放类型',
126567
126568   -- 时间限制
126569   `valid_days` INT DEFAULT 30 COMMENT '有效天数（从领取时开始计算）',
126570   `fixed_valid_start` DATETIME COMMENT '固定有效开始时间',
126571   `fixed_valid_end` DATETIME COMMENT '固定有效结束时间',
126572
126573   -- 使用限制
126574   `max_usage_total` INT DEFAULT NULL COMMENT '总使用次数限制',
126575   `max_usage_per_user` INT DEFAULT 1 COMMENT '每用户使用次数限制',
126576   `first_purchase_only` BOOLEAN DEFAULT FALSE COMMENT '仅限首次购买',
126577   `new_user_only` BOOLEAN DEFAULT FALSE COMMENT '仅限新用户',
126578
126579   -- 状态管理
126580   `status` ENUM('draft', 'active', 'paused', 'expired') DEFAULT 'draft' COMMENT '状态',
126581   `priority` INT DEFAULT 0 COMMENT '优先级',
126582
126583   -- 邀请相关
126584   `is_invite_coupon` BOOLEAN DEFAULT FALSE COMMENT '是否为邀请折扣券',
126585   `inviter_benefit` JSON COMMENT '邀请人奖励配置',
126586
126587   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126588   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
126589   `created_by` BIGINT UNSIGNED COMMENT '创建人ID',
126590
126591   PRIMARY KEY (`id`),
126592   UNIQUE KEY `uk_code` (`code`),
126593   INDEX `idx_status_type` (`status`, `distribution_type`),
126594   INDEX `idx_valid_time` (`fixed_valid_start`, `fixed_valid_end`)
126595 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='折扣券模板表';
126596
126597 -- 用户折扣券表
126598 CREATE TABLE IF NOT EXISTS `user_coupons` (
126599   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126600   `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID',
126601   `template_id` BIGINT UNSIGNED NOT NULL COMMENT '折扣券模板ID',
126602   `coupon_code` VARCHAR(100) COMMENT '折扣券唯一码',
126603
126604   -- 来源追踪
126605   `source_type` ENUM('manual', 'invite', 'register', 'batch', 'targeted', 'activity') NOT NUL
L COMMENT '获取方式',
126606   `source_id` VARCHAR(100) COMMENT '来源ID',
126607   `source_detail` JSON COMMENT '来源详情',
126608
126609   -- 有效期
126610   `valid_from` DATETIME NOT NULL COMMENT '生效时间',
126611   `valid_until` DATETIME NOT NULL COMMENT '失效时间',
126612
126613   -- 使用状态
126614   `status` ENUM('unused', 'reserved', 'used', 'expired', 'cancelled') DEFAULT 'unused' COMMEN
T '状态: unused=未使用, reserved=已预留, used=已使用, expired=已过期, cancelled=已取消',
126615   `used_at` DATETIME COMMENT '使用时间',
126616   `used_order_id` BIGINT UNSIGNED COMMENT '使用订单ID',
126617
126618   -- 折扣信息（冗余存储）
126619   `discount_type` VARCHAR(20) NOT NULL COMMENT '折扣类型',
126620   `discount_value` DECIMAL(10,2) NOT NULL COMMENT '折扣值',
126621   `min_purchase_amount` DECIMAL(10,2) DEFAULT 0 COMMENT '最低消费金额',
126622
126623   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126624   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126625
126626   PRIMARY KEY (`id`),
126627   UNIQUE KEY `uk_coupon_code` (`coupon_code`),
126628   INDEX `idx_user_status` (`user_id`, `status`),
126629   INDEX `idx_template` (`template_id`),
126630   INDEX `idx_valid_time` (`valid_from`, `valid_until`),
126631   INDEX `idx_source` (`source_type`, `source_id`),
126632   CONSTRAINT `fk_user_coupon_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELET
E CASCADE,
126633   CONSTRAINT `fk_user_coupon_template` FOREIGN KEY (`template_id`) REFERENCES `coupon_templat
es` (`id`)
126634 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户折扣券表';
126635
126636 -- 折扣券使用记录表
126637 CREATE TABLE IF NOT EXISTS `coupon_usage_logs` (
126638   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126639   `user_coupon_id` BIGINT UNSIGNED NOT NULL COMMENT '用户折扣券ID',
126640   `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID',
126641   `order_id` BIGINT UNSIGNED NOT NULL COMMENT '订单ID',
```

```
126642
126643  `original_amount` DECIMAL(10,2) NOT NULL COMMENT '原始金额',
126644  `discount_amount` DECIMAL(10,2) NOT NULL COMMENT '折扣金额',
126645  `final_amount` DECIMAL(10,2) NOT NULL COMMENT '最终金额',
126646
126647  `status` ENUM('applied', 'confirmed', 'refunded') DEFAULT 'applied' COMMENT '状态',
126648
126649  `applied_at` DATETIME DEFAULT CURRENT_TIMESTAMP COMMENT '应用时间',
126650  `confirmed_at` DATETIME COMMENT '确认时间',
126651  `refunded_at` DATETIME COMMENT '退款时间',
126652
126653  `metadata` JSON COMMENT '额外信息',
126654
126655  PRIMARY KEY (`id`),
126656  INDEX `idx_user_coupon` (`user_coupon_id`),
126657  INDEX `idx_user` (`user_id`),
126658  INDEX `idx_order` (`order_id`),
126659  CONSTRAINT `fk_usage_log_user_coupon` FOREIGN KEY (`user_coupon_id`) REFERENCES `user_coupo
ns` (`id`),
126660  CONSTRAINT `fk_usage_log_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE
CASCADE,
126661  CONSTRAINT `fk_usage_log_order` FOREIGN KEY (`order_id`) REFERENCES `orders` (`id`) ON DELE
TE CASCADE
126662 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='折扣券使用记录表';
126663
126664 -- 用户邀请表（优化版）
126665 CREATE TABLE IF NOT EXISTS `user_invitations` (
126666  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126667  `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID',
126668  `invitation_code` VARCHAR(50) NOT NULL COMMENT '邀请码',
126669  `invitation_url` VARCHAR(500) COMMENT '邀请链接',
126670
126671  -- 统计数据
126672  `total_invites` INT DEFAULT 0 COMMENT '总邀请数',
126673  `successful_invites` INT DEFAULT 0 COMMENT '成功邀请数（完成注册）',
126674  `paid_invites` INT DEFAULT 0 COMMENT '付费邀请数（完成首购）',
126675
126676  -- 状态
126677  `is_active` BOOLEAN DEFAULT TRUE COMMENT '是否激活',
126678
126679  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126680  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126681
126682  PRIMARY KEY (`id`),
126683  UNIQUE KEY `uk_user_id` (`user_id`),
126684  UNIQUE KEY `uk_invitation_code` (`invitation_code`),
126685  INDEX `idx_is_active` (`is_active`),
126686  CONSTRAINT `fk_invitation_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE
 CASCADE
126687 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户邀请信息表';
126688
126689 -- 邀请关系表
126690 CREATE TABLE IF NOT EXISTS `invitation_relationships` (
126691  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126692  `inviter_id` BIGINT UNSIGNED NOT NULL COMMENT '邀请人用户ID',
126693  `invitee_id` BIGINT UNSIGNED NOT NULL COMMENT '被邀请人用户ID',
126694  `invitation_code` VARCHAR(50) COMMENT '使用的邀请码',
```

```
126695
126696  `status` ENUM('pending', 'registered', 'activated', 'rewarded') DEFAULT 'pending' COMMENT '
状态',
126697
126698  `invitee_coupon_id` BIGINT UNSIGNED COMMENT '被邀请人获得的折扣券ID',
126699  `inviter_reward_id` BIGINT UNSIGNED COMMENT '邀请人奖励ID',
126700
126701  `registered_at` DATETIME COMMENT '注册时间',
126702  `activated_at` DATETIME COMMENT '激活时间（首次付费）',
126703  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126704
126705  PRIMARY KEY (`id`),
126706  UNIQUE KEY `uk_invitee` (`invitee_id`),
126707  INDEX `idx_inviter` (`inviter_id`),
126708  INDEX `idx_invitation_code` (`invitation_code`),
126709  INDEX `idx_status` (`status`),
126710  CONSTRAINT `fk_relationship_inviter` FOREIGN KEY (`inviter_id`) REFERENCES `users` (`id`) O
N DELETE CASCADE,
126711  CONSTRAINT `fk_relationship_invitee` FOREIGN KEY (`invitee_id`) REFERENCES `users` (`id`) O
N DELETE CASCADE
126712 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='邀请关系表';
126713
126714 -- 折扣券批次表
126715 CREATE TABLE IF NOT EXISTS `coupon_batches` (
126716  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126717  `batch_code` VARCHAR(50) NOT NULL COMMENT '批次号',
126718  `template_id` BIGINT UNSIGNED NOT NULL COMMENT '折扣券模板ID',
126719  `name` VARCHAR(100) NOT NULL COMMENT '批次名称',
126720  `description` TEXT COMMENT '批次描述',
126721
126722  -- 发放规则
126723  `target_type` ENUM('all', 'new_users', 'paid_users', 'specific_users', 'user_group') NOT NU
LL COMMENT '目标用户类型',
126724  `target_users` JSON COMMENT '特定用户ID列表',
126725  `target_conditions` JSON COMMENT '用户筛选条件',
126726
126727  `total_quantity` INT NOT NULL COMMENT '发放总量',
126728  `distributed_quantity` INT DEFAULT 0 COMMENT '已发放数量',
126729  `used_quantity` INT DEFAULT 0 COMMENT '已使用数量',
126730
126731  `status` ENUM('draft', 'processing', 'completed', 'cancelled') DEFAULT 'draft',
126732
126733  `scheduled_at` DATETIME COMMENT '计划发放时间',
126734  `started_at` DATETIME COMMENT '开始发放时间',
126735  `completed_at` DATETIME COMMENT '完成时间',
126736
126737  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126738  `created_by` BIGINT UNSIGNED COMMENT '创建人ID',
126739
126740  PRIMARY KEY (`id`),
126741  UNIQUE KEY `uk_batch_code` (`batch_code`),
126742  INDEX `idx_template` (`template_id`),
126743  INDEX `idx_status` (`status`),
126744  CONSTRAINT `fk_batch_template` FOREIGN KEY (`template_id`) REFERENCES `coupon_templates` (`
id`)
126745 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='折扣券批次表';
126746
```

```
126747 -- ============================================
126748 -- 系统配置表
126749 -- ============================================
126750
126751 -- 隐私政策
126752 CREATE TABLE IF NOT EXISTS `privacy_policies` (
126753   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126754   `content` TEXT NOT NULL,
126755   `version` VARCHAR(50) NOT NULL,
126756   `is_active` BOOLEAN NOT NULL DEFAULT TRUE,
126757   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126758   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126759   PRIMARY KEY (`id`),
126760   INDEX `idx_is_active` (`is_active`),
126761   INDEX `idx_created_at` (`created_at`)
126762 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='隐私政策';
126763
126764 -- 服务条款
126765 CREATE TABLE IF NOT EXISTS `terms_of_service` (
126766   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126767   `content` TEXT NOT NULL,
126768   `version` VARCHAR(50) NOT NULL,
126769   `is_active` BOOLEAN NOT NULL DEFAULT TRUE,
126770   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126771   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126772   PRIMARY KEY (`id`),
126773   INDEX `idx_is_active` (`is_active`),
126774   INDEX `idx_created_at` (`created_at`)
126775 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='服务条款';
126776
126777 -- 邮件配置
126778 CREATE TABLE IF NOT EXISTS `email_configs` (
126779   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126780   `smtp_host` VARCHAR(255) NOT NULL,
126781   `smtp_port` INT NOT NULL,
126782   `smtp_user` VARCHAR(255) NOT NULL,
126783   `smtp_password` VARCHAR(255) NOT NULL,
126784   `from_email` VARCHAR(255) NOT NULL,
126785   `from_name` VARCHAR(255) NOT NULL,
126786   `is_active` BOOLEAN NOT NULL DEFAULT TRUE,
126787   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126788   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126789   PRIMARY KEY (`id`),
126790   INDEX `idx_is_active` (`is_active`),
126791   INDEX `idx_created_at` (`created_at`)
126792 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='邮件服务器配置';
126793
126794 -- 短信配置（阿里云）
126795 CREATE TABLE IF NOT EXISTS `sms_configs` (
126796   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126797   `access_key_id` VARCHAR(255) NOT NULL COMMENT '阿里云Access Key ID',
126798   `access_secret` VARCHAR(255) NOT NULL COMMENT '阿里云Access Secret',
126799   `sign_name` VARCHAR(100) NOT NULL COMMENT '短信签名',
126800   `template_code` VARCHAR(50) NOT NULL COMMENT '短信模板代码',
126801   `template_content` TEXT NOT NULL COMMENT '短信模板内容',
126802   `endpoint` VARCHAR(255) NOT NULL DEFAULT 'dypnsapi.aliyuncs.com',
126803   `is_active` BOOLEAN NOT NULL DEFAULT TRUE,
```

```
126804   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126805   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126806   PRIMARY KEY (`id`),
126807   INDEX `idx_is_active` (`is_active`),
126808   INDEX `idx_created_at` (`created_at`)
126809 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='短信服务配置';
126810
126811 -- 支付宝配置
126812 CREATE TABLE IF NOT EXISTS `alipay_configs` (
126813   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126814   `app_id` VARCHAR(32) NOT NULL COMMENT '支付宝App ID',
126815   `private_key` TEXT NOT NULL COMMENT '应用私钥',
126816   `alipay_public_key` TEXT NOT NULL COMMENT '支付宝公钥',
126817   `gateway_url` VARCHAR(255) NOT NULL DEFAULT 'https://openapi.alipay.com/gateway.do',
126818   `notify_url` VARCHAR(255) NOT NULL COMMENT '支付回调URL',
126819   `return_url` VARCHAR(255) COMMENT '支付后返回URL',
126820   `is_sandbox` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '是否沙箱环境',
126821   `is_active` BOOLEAN NOT NULL DEFAULT TRUE,
126822   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126823   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126824   PRIMARY KEY (`id`),
126825   INDEX `idx_is_active` (`is_active`)
126826 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='支付宝配置';
126827
126828 -- 微信支付配置
126829 CREATE TABLE IF NOT EXISTS `wechat_pay_configs` (
126830   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126831   `mch_id` VARCHAR(32) NOT NULL COMMENT '商户号',
126832   `app_id` VARCHAR(32) NOT NULL COMMENT '微信App ID',
126833   `api_v3_key` VARCHAR(64) NOT NULL COMMENT 'APIv3密钥',
126834   `serial_no` VARCHAR(64) NOT NULL COMMENT '商户API证书序列号',
126835   `private_key` TEXT NOT NULL COMMENT '商户私钥',
126836   `notify_url` VARCHAR(255) NOT NULL COMMENT '支付回调URL',
126837   `is_sandbox` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '是否沙箱环境',
126838   `is_active` BOOLEAN NOT NULL DEFAULT TRUE,
126839   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126840   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126841   PRIMARY KEY (`id`),
126842   INDEX `idx_is_active` (`is_active`)
126843 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='微信支付配置';
126844
126845 -- 微信扫码登录配置（原微信OAuth配置）
126846 CREATE TABLE IF NOT EXISTS `wechat_configs` (
126847   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126848   `app_id` VARCHAR(32) NOT NULL COMMENT '微信App ID',
126849   `app_secret` VARCHAR(64) NOT NULL COMMENT '微信App Secret',
126850   `redirect_uri` VARCHAR(255) NOT NULL COMMENT 'OAuth重定向URI',
126851   `scope` VARCHAR(50) NOT NULL DEFAULT 'snsapi_login',
126852   `is_active` BOOLEAN NOT NULL DEFAULT TRUE,
126853   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126854   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126855   PRIMARY KEY (`id`),
126856   INDEX `idx_is_active` (`is_active`)
126857 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='微信OAuth配置';
126858
126859 -- 系统配置
126860 CREATE TABLE IF NOT EXISTS `system_configs` (
```

```
126861  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126862  `require_email_verification` BOOLEAN NOT NULL DEFAULT TRUE COMMENT '是否需要邮箱验证',
126863  `enable_turnstile` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '是否启用 Cloudflare Turnstile 人机验证',
126864  `turnstile_site_key` VARCHAR(100) DEFAULT '' COMMENT 'Turnstile Site Key (公钥)',
126865  `turnstile_secret_key` VARCHAR(100) DEFAULT '' COMMENT 'Turnstile Secret Key (私钥)',
126866  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126867  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126868  PRIMARY KEY (`id`)
126869 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='系统配置';
126870
126871 -- GitHub OAuth 配置表（独立表）
126872 CREATE TABLE IF NOT EXISTS `github_oauth_config` (
126873  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126874  `client_id` VARCHAR(100) NOT NULL DEFAULT '' COMMENT 'GitHub OAuth Client ID',
126875  `client_secret` VARCHAR(255) NOT NULL DEFAULT '' COMMENT 'GitHub OAuth Client Secret',
126876  `redirect_uri` VARCHAR(255) NOT NULL DEFAULT '' COMMENT 'GitHub OAuth Redirect URI (回调地址)',
126877  `enabled` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '是否启用 GitHub OAuth 登录',
126878  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126879  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126880  PRIMARY KEY (`id`)
126881 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='GitHub OAuth 配置';
126882
126883 -- 微信公众号自动回复配置
126884 CREATE TABLE IF NOT EXISTS `wechat_official_config` (
126885  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126886  `app_id` VARCHAR(100) NOT NULL DEFAULT '' COMMENT '微信公众号AppID',
126887  `app_secret` VARCHAR(255) NOT NULL DEFAULT '' COMMENT '微信公众号AppSecret',
126888  `token` VARCHAR(100) NOT NULL DEFAULT '' COMMENT '微信公众号Token',
126889  `encoding_aes_key` VARCHAR(100) NOT NULL DEFAULT '' COMMENT '微信公众号EncodingAESKey',
126890  `encryption_mode` VARCHAR(20) NOT NULL DEFAULT 'plaintext' COMMENT '消息加密方式: plaintext=明文模式,
 compatible=兼容模式, secure=安全模式',
126891  `auto_reply_enabled` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '是否启用自动回复',
126892  `ai_model_level` VARCHAR(20) NOT NULL DEFAULT 'low' COMMENT 'AI模型等级: low, medium, high',
126893  `system_prompt` TEXT COMMENT 'AI系统提示词',
126894  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126895  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126896  PRIMARY KEY (`id`)
126897 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='微信公众号自动回复配置';
126898
126899 -- 第三方账号配置
126900 CREATE TABLE IF NOT EXISTS `third_party_accounts` (
126901  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126902  `name` VARCHAR(100) NOT NULL COMMENT '账号名称/描述',
126903  `type` VARCHAR(50) NOT NULL COMMENT '账号类型: semantic_scholar, ai_model',
126904  `config` JSON NOT NULL COMMENT '配置详情',
126905  `status` TINYINT NOT NULL DEFAULT 1 COMMENT '1=启用, 0=停用',
126906  `priority` INT NOT NULL DEFAULT 0 COMMENT '优先级，数字越小优先级越高',
126907  `usage_count` BIGINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '使用次数',
126908  `last_used_at` DATETIME COMMENT '最后使用时间',
126909  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126910  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126911  PRIMARY KEY (`id`),
126912  INDEX `idx_type_status` (`type`, `status`),
126913  INDEX `idx_type_priority` (`type`, `status`, `priority`),
126914  INDEX `idx_last_used` (`last_used_at`)
126915 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='第三方API账号配置';
126916
```

```
126917 -- 提示词模板
126918 CREATE TABLE IF NOT EXISTS `prompt_templates` (
126919   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126920   `name` VARCHAR(100) NOT NULL COMMENT '模板名称',
126921   `key` VARCHAR(50) NOT NULL COMMENT '模板键值',
126922   `template` TEXT NOT NULL COMMENT '提示词模板',
126923   `description` TEXT COMMENT '模板描述',
126924   `variables` JSON COMMENT '模板变量定义',
126925   `is_active` TINYINT NOT NULL DEFAULT 1 COMMENT '1=启用, 0=停用',
126926   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126927   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126928   PRIMARY KEY (`id`),
126929   UNIQUE INDEX `uk_key` (`key`),
126930   INDEX `idx_active` (`is_active`)
126931 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='AI提示词模板';
126932
126933 -- 论文AI分析结果缓存表（动态维度设计）
126934 -- 同一篇论文可以有多条记录，对应不同的维度配置（dimension_hash）
126935 CREATE TABLE IF NOT EXISTS `paper_analysis` (
126936   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
126937   `paper_id` VARCHAR(100) NOT NULL COMMENT '论文ID（来自Semantic Scholar）',
126938   `paper_title` VARCHAR(500) COMMENT '论文标题',
126939   `dimension_hash` VARCHAR(32) NOT NULL COMMENT '维度配置哈希，用于判断缓存是否匹配当前维度配置',
126940   `analysis_data` TEXT COMMENT 'JSON格式的分析结果 {"dimensionKey": "分析内容", ...}',
126941   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
126942   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
126943   PRIMARY KEY (`id`),
126944   UNIQUE INDEX `uk_paper_dimension` (`paper_id`, `dimension_hash`),
126945   INDEX `idx_dimension_hash` (`dimension_hash`),
126946   INDEX `idx_created_at` (`created_at`)
126947 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='论文AI分析结果缓存（动态
;
126948
126949 SET foreign_key_checks = 1;
126950
126951 -- ============================================
126952 -- 预置数据（仅在表不存在数据时插入）
126953 -- 使用INSERT IGNORE确保不会覆盖已有数据
126954 -- ============================================
126955
126956 -- 默认管理员账号由后端服务启动时自动初始化（从环境变量读取）
126957 -- 配置环境变量: ADMIN_INITIAL_USERNAME, ADMIN_INITIAL_PASSWORD
126958 -- 详见 backend/internal/admin/service.go InitializeDefaultAdmin()
126959
126960 -- 默认订阅计划
126961 INSERT IGNORE INTO `subscription_plans` (`plan_code`, `plan_name`, `price_monthly`, `current_
price_monthly`, `price_yearly`, `current_price_yearly`, `yearly_discount_enabled`, `yearly_discount_
percent`, `features`, `is_active`, `sort_order`) VALUES
126962 ('free', 'FREE', 0.00, 0.00, 0.00, 0.00, 0, 0.00, '{
126963   "normal_search": {"limit": -1, "description": "无限制"},
126964   "advanced_search_low": {"limit": 5, "period": "day", "description": "5/天"},
126965   "advanced_search_mid": {"limit": 3, "period": "day", "description": "3/天"},
126966   "advanced_search_high": {"limit": 2, "period": "day", "description": "2/天"},
126967   "ai_chat_normal": {"limit": -1, "description": "无限制"},
126968   "ai_chat_mid": {"limit": 50, "period": "day", "description": "50/天"},
126969   "ai_chat_high": {"limit": 10, "period": "day", "description": "10/天"},
126970   "advanced_search_data": {"enabled": true, "description": "无限制"},
```

```
126971   "ai_history_data": {"enabled": true, "description": "无限制"}
126972 }', 1, 1),
126973 ('pro', 'PRO', 88.00, 88.00, 880.00, 880.00, 0, 0.00, '{
126974   "normal_search": {"limit": -1, "description": "无限制"},
126975   "advanced_search_low": {"limit": -1, "description": "无限制"},
126976   "advanced_search_mid": {"limit": -1, "description": "无限制"},
126977   "advanced_search_high": {"limit": -1, "description": "无限制"},
126978   "ai_chat_normal": {"limit": -1, "description": "无限制"},
126979   "ai_chat_mid": {"limit": -1, "description": "无限制"},
126980   "ai_chat_high": {"limit": -1, "description": "无限制"},
126981   "advanced_search_data": {"enabled": true, "description": "无限制"},
126982   "ai_history_data": {"enabled": true, "description": "无限制"}
126983 }', 1, 2),
126984 ('max', 'MAX', 199.00, 199.00, 1990.00, 1990.00, 0, 0.00, '{
126985   "normal_search": {"limit": -1, "description": "无限制"},
126986   "advanced_search_low": {"limit": -1, "description": "无限制"},
126987   "advanced_search_mid": {"limit": -1, "description": "无限制"},
126988   "advanced_search_high": {"limit": -1, "description": "无限制"},
126989   "ai_chat_normal": {"limit": -1, "description": "无限制"},
126990   "ai_chat_mid": {"limit": -1, "description": "无限制"},
126991   "ai_chat_high": {"limit": -1, "description": "无限制"},
126992   "advanced_search_data": {"enabled": true, "description": "无限制"},
126993   "ai_history_data": {"enabled": true, "description": "无限制"}
126994 }', 1, 3);
126995
126996 -- 默认提示词模板
126997 -- 设置字符集以确保中文正确存储
126998 SET NAMES utf8mb4 COLLATE utf8mb4_unicode_ci;
126999
127000 INSERT IGNORE INTO `prompt_templates` (`key`, `name`, `template`, `description`, `variables`,
`is_active`) VALUES
127001 ('advanced_search_mcp_system', '高级搜索MCP工具调用系统提示词',
127002 '你是一位专业的学术研究助手,专门帮助用户搜索和分析学术论文。
127003
127004 **核心任务**:
127005 - 这是一个学术论文搜索系统,用户的每个查询都是为了找到相关的学术论文
127006 - 你必须使用 search_papers 工具来搜索论文,即使用户的查询看起来像是在问概念或问题
127007 - 不要直接回答问题或解释概念,而是搜索相关论文
127008
127009 **工具调用规则**:
127010 1. 对于所有用户查询,首先调用 search_papers 工具搜索论文
127011 2. 将用户的查询转换为合适的英文学术搜索关键词(因为Semantic Scholar主要收录英文论文)
127012 3. 设置 limit 参数为 40,返回更多相关论文
127013 4. 可以提取查询中的关键概念作为搜索词,例如:
127014    - \"大语言模型的算力需求\" → \"large language models computational requirements GPU\"
127015    - \"向量搜索算法\" → \"vector search algorithms\"
127016    - \"Transformer架构\" → \"Transformer architecture\"
127017
127018 **搜索策略**:
127019 - 优先使用核心技术术语和英文关键词
127020 - 如果查询包含多个概念,组合关键概念进行搜索
127021 - 确保搜索词足够具体以找到相关论文
127022
127023 请务必调用工具,不要直接回答!',
127024 '引导AI必须使用search_papers工具搜索论文,并将中文查询转换为英文学术关键词',
127025 NULL,
127026 1),
```

127027
127028 ('advanced_search_analysis_system', '高级搜索分析系统提示词',
127029 '你是一位专业的学术研究助手，擅长分析和总结学术论文。',
127030 '用于高级搜索论文分析时定义AI的专业角色',
127031 NULL,
127032 1),
127033
127034 ('advanced_search_analysis', '高级搜索深度论文分析提示词',
127035 '用户查询：{{query}}
127036
127037 找到的论文：
127038 {{papers}}
127039
127040 请作为资深学术研究专家，对这些论文进行深入、全面的分析。你的分析应该包括：
127041
127042 ## 1. 研究领域概览（200-300字）
127043 - 这些论文所属的主要研究领域和细分方向
127044 - 该领域当前的研究热点和发展趋势
127045 - 这些论文在整个领域中的定位和意义
127046
127047 ## 2. 核心研究方法与创新点（按论文分析，每篇100-150字）
127048 - 每篇论文采用的核心研究方法和技术路线
127049 - 相比前人工作的创新之处（方法创新、应用创新、理论创新）
127050 - 技术实现的关键突破点和独特优势
127051 - 实验设计的亮点和数据支撑的充分性
127052
127053 ## 3. 研究发现与学术贡献（深度分析）
127054 - 各论文的核心发现和主要结论
127055 - 对现有理论或认知的突破或修正
127056 - 实证数据和实验结果的说服力
127057 - 研究结果的普适性和局限性
127058 - 不同论文之间的相互印证或矛盾之处
127059
127060 ## 4. 实际应用价值与影响
127061 - 研究成果的实际应用场景和商业价值
127062 - 对产业发展或社会问题的潜在影响
127063 - 技术转化的可行性和实施难度
127064 - 已产生的实际应用案例（如有）
127065
127066 ## 5. 研究脉络与发展演进
127067 - 论文之间的引用关系和研究传承
127068 - 研究思路的演进轨迹和方法论发展
127069 - 从早期到近期研究的进步和突破
127070 - 研究范式的转变和新兴方向
127071
127072 ## 6. 批判性思考与研究展望
127073 - 现有研究的局限性和不足之处
127074 - 尚未解决的核心问题和研究空白
127075 - 未来可能的研究方向和突破点
127076 - 跨学科融合的可能性
127077
127078 重要提示：
127079 1. **引用格式（非常重要）**：当提到具体论文的观点、数据或结论时，必须使用以下格式：
127080 【引用编号：引用的具体内容】
127081
127082 正确示例：
127083 - 【引用1：深度学习模型在ImageNet数据集上达到了95%%的准确率】表明该方法效果显著。

```
127084   - 研究表明，【引用3：Transformer架构相比CNN减少了40%%的训练时间】。
127085
127086   错误示例（不要这样做）：
127087   - 论文1指出深度学习效果很好 [1]
127088   - 根据研究[3]，效果提升明显
127089
127090   **每个段落至少使用3-5个引用标记**，确保分析有具体的论文支撑。
127091
127092 2. 分析要深入且有洞察力，避免表面化和泛泛而谈
127093 3. 多引用论文中的具体数据、实验结果和关键结论（使用【引用n：内容】格式）
127094 4. 保持客观和批判性思维，指出优点的同时也要指出不足
127095
127096 请用 Markdown 格式输出完整的分析报告。注意：不要包含延伸思考问题，只需要输出以上6个部分的分析。',
127097 '用于高级搜索功能中生成深度、全面的论文分析报告，包含6个分析维度（不含延伸问题）',
127098 '{"query": {"type": "string", "description": "User search query", "example": "Transformer in
NLP"}, "papers": {"type": "string", "description": "Formatted paper list with number, title, authors
, abstract, citations", "example": "1. Title: Attention Is All You Need, Authors: Vaswani et al., Ye
ar: 2017"}}',
127099 1),
127100
127101 ('follow_up_questions_system', '延伸问题生成系统提示词',
127102 '你是一位专业的学术研究助手，擅长基于研究分析提出有深度的延伸问题。',
127103 '用于生成延伸思考问题时定义AI的角色',
127104 NULL,
127105 1),
127106
127107 ('follow_up_questions', '延伸思考问题生成提示词',
127108 '用户查询：{{query}}
127109
127110 已完成的分析：
127111 {{analysis}}
127112
127113 基于上述分析，请提供3-5个发散性的后续研究问题，格式如下：
127114 [FOLLOW_UP_QUESTIONS]
127115 1. 问题文本1
127116 2. 问题文本2
127117 3. 问题文本3
127118 [/FOLLOW_UP_QUESTIONS]
127119
127120 这些问题应该：
127121 - 基于当前研究的自然延伸
127122 - 具有一定的探索性和前瞻性
127123 - 涵盖不同角度（技术深化、应用拓展、跨领域融合、伦理思考等）
127124 - 能够引发深入讨论
127125 - 每个问题都应该简洁明了，一句话即可
127126
127127 请直接输出问题列表，不要有其他内容。',
127128 '用于基于分析内容生成延伸思考问题，使用low模型独立调用',
127129 '{"query": {"type": "string", "description": "User query", "example": "Transformer架构"}, "anal
ysis": {"type": "string", "description": "Completed analysis content", "example": "## 研究领域概览\\n..."}
}',
127130 1);
127131
127132 -- ============================================
127133 -- 论文分析维度表
127134 -- ============================================
127135
```

```
-- 分析维度定义表（支持系统默认和用户自定义）
-- user_id = -1 表示系统默认维度，user_id > 0 表示用户自定义维度
CREATE TABLE IF NOT EXISTS `analysis_dimensions` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `user_id` BIGINT NOT NULL DEFAULT -1 COMMENT '用户ID，-1表示系统默认维度，>0表示用户自定义',
  `dimension_key` VARCHAR(50) NOT NULL COMMENT '维度标识符（如 studyType, bigDataFocus）',
  `name` VARCHAR(100) NOT NULL COMMENT '维度名称（显示用）',
  `description` TEXT NOT NULL COMMENT '维度解释/分析指示',
  `sort_order` INT NOT NULL DEFAULT 0 COMMENT '排序顺序',
  `is_active` BOOLEAN NOT NULL DEFAULT TRUE COMMENT '是否启用',
  `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE INDEX `uk_user_dimension` (`user_id`, `dimension_key`),
  INDEX `idx_user_id` (`user_id`),
  INDEX `idx_dimension_key` (`dimension_key`),
  INDEX `idx_sort_order` (`sort_order`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='论文分析维度定义表';

-- 插入系统默认的8个分析维度（user_id = -1 表示系统级）
INSERT IGNORE INTO `analysis_dimensions` (`user_id`, `dimension_key`, `name`, `description`,
`sort_order`) VALUES
(-1, 'studyType', '研究类型', '请判断论文属于以下哪类（可多选，缺乏证据则标注“未提及”）：
- 原创实证研究（定量 / 定性）
- 实验研究（biology/chemistry/psychology/engineering）
- 方法论文 / 模型论文（Method / Model）
- 理论或概念框架论文（Theoretical / Conceptual）
- 文献综述（Narrative Review）
- 系统综述 / 元分析（Systematic Review / Meta-analysis）
- 调查 / 教程 / 技术综述（Survey / Tutorial）
- 案例研究（Case Study）
- 观点 / 评论类文章（Opinion / Commentary）
需要根据摘要中的显性行为判断，如是否出现“we propose”、“we review”、“we analyze experiments”等。
若摘要未提供相关信息，请输出 “未提及”。', 1),
(-1, 'researchObjectives', '研究目的与核心问题', '提取论文试图解决的核心科学问题或实践问题，包括：
- 研究动机
- 想回答的问题
- 想解决的挑战
若摘要存在直接表述（例如 "This study aims to"），请引用关键信息。
若摘要未提供相关信息，请输出 “未提及”。', 2),
(-1, 'researchObject', '研究对象与应用场景', '请识别论文研究作用于哪些对象，包括但不限于：
- 人群 / 组织 / 社会群体
- 技术系统 / 模型 / 算法 / 数据库
- 物理对象 / 生物材料 / 地理区域
- 某种现象或过程
若适用，请描述论文的应用场景（如发展中国家、医疗领域、教育系统等）。
若摘要未提供相关信息，请输出 “未提及”。', 3),
(-1, 'researchMethods', '研究方法/技术路径', '请总结论文的方法论，包括：
- 定量/定性方法
- 实验、模拟、建模、统计分析
- 文献分析策略（若为综述）
- 多案例研究方法
只基于摘要中明确提及的内容，不可推断未出现的方法。
若摘要未提供相关信息，请输出 “未提及”。', 4),
(-1, 'dataMaterialSources', '数据与材料来源', '请说明论文使用的数据类型或材料，包括：
- 真实数据、实验数据、测量数据、调查数据
- 文献样本（用于综述）
```

```
127192 - 公共数据集、合成数据
127193 若摘要未提到数据来源，请明确标注“未提及”。', 5),
127194 (-1, 'keyFindingsRresults', '主要发现/关键结果', '总结论文得出的核心发现或结果，例如：
127195 - 理论推论
127196 - 实验结果
127197 - 性能改进
127198 - 对既有文献的总结性洞察
127199 必须基于摘要中真实出现的信息。
127200 若摘要未提供相关信息，请输出 “未提及”。', 6),
127201 (-1, 'innovationContribution', '创新点/贡献', '提取论文相对既有研究的创新性，包括：
127202 - 理论创新
127203 - 方法创新
127204 - 实证贡献
127205 - 整合型贡献（如提出新分类、新框架、新 taxonomy）
127206 若论文未明确指出创新点，请写“未提及”。', 7),
127207 (-1, 'practicalSignificanceApplication', '实际意义/应用价值', '说明研究的潜在影响，包括：
127208 - 理论价值（推动学科发展）
127209 - 实践价值（改善流程/技术/政策）
127210 - 社会/经济/行业意义
127211 - 对未来研究的指引
127212 若摘要仅隐含意义，请根据显性内容谨慎归纳。
127213 若摘要未提供相关信息，请输出 “未提及”。', 8);
127214
127215 -- ============================================
127216 -- AI审稿人系统表
127217 -- ============================================
127218
127219 -- 审稿任务表（主表）
127220 CREATE TABLE IF NOT EXISTS `peer_review_tasks` (
127221   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
127222   `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID',
127223   `input_type` VARCHAR(20) NOT NULL DEFAULT 'text' COMMENT '输入类型: text=文本, file=文件',
127224   `manuscript_text` LONGTEXT COMMENT '稿件文本内容',
127225   `file_id` BIGINT UNSIGNED COMMENT '关联的文件ID',
127226   `file_name` VARCHAR(500) COMMENT '上传的文件名',
127227   `manuscript_hash` VARCHAR(64) COMMENT '内容哈希，用于缓存',
127228   `title` VARCHAR(500) COMMENT '稿件标题（从内容中提取）',
127229   `target_venue` VARCHAR(255) COMMENT '目标期刊/会议',
127230   `model_level` VARCHAR(20) DEFAULT 'mid' COMMENT '模型级别: low, mid, high',
127231
127232   -- 任务状态
127233   `status` VARCHAR(20) NOT NULL DEFAULT 'pending' COMMENT '状态: pending=等待处理, processing=处理中,
completed=已完成, failed=失败',
127234   `progress` INT NOT NULL DEFAULT 0 COMMENT '进度百分比 0-100',
127235   `progress_stage` VARCHAR(100) COMMENT '当前阶段描述',
127236   `error_message` TEXT COMMENT '错误信息',
127237
127238   -- 处理结果（JSON存储）
127239   `manuscript_meta` TEXT COMMENT '稿件元信息 JSON',
127240   `literature_context` TEXT COMMENT '文献上下文 JSON',
127241   `literature_summary` TEXT COMMENT '文献总结 JSON',
127242   `gap_literature` TEXT COMMENT '研究空白相关文献 JSON',
127243   `review_content` LONGTEXT COMMENT 'Markdown格式的审稿报告',
127244   `recommendation` VARCHAR(50) COMMENT '最终推荐: Accept, Minor Revision, Major Revision, Reject'
,
127245
127246   -- 时间戳
```

```
127247   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127248   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
127249   `started_at` DATETIME COMMENT '开始处理时间',
127250   `completed_at` DATETIME COMMENT '完成时间',
127251
127252   PRIMARY KEY (`id`),
127253   INDEX `idx_user_id` (`user_id`),
127254   INDEX `idx_status` (`status`),
127255   INDEX `idx_user_status` (`user_id`, `status`),
127256   INDEX `idx_user_created` (`user_id`, `created_at` DESC),
127257   INDEX `idx_manuscript_hash` (`manuscript_hash`),
127258   CONSTRAINT `fk_peer_review_task_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON
DELETE CASCADE
127259 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='AI审稿任务表';
127260
127261 -- 审稿文件表（存储上传的文件）
127262 CREATE TABLE IF NOT EXISTS `peer_review_files` (
127263   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
127264   `user_id` BIGINT UNSIGNED NOT NULL COMMENT '用户ID',
127265   `file_name` VARCHAR(255) NOT NULL COMMENT '原始文件名',
127266   `file_type` VARCHAR(50) COMMENT '文件类型: docx, doc, txt, pdf',
127267   `file_size` BIGINT COMMENT '文件大小（字节）',
127268   `file_path` VARCHAR(500) COMMENT '存储路径（可选）',
127269   `extracted_text` LONGTEXT COMMENT '提取的文本内容',
127270   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127271
127272   PRIMARY KEY (`id`),
127273   INDEX `idx_user_id` (`user_id`),
127274   INDEX `idx_created_at` (`created_at`),
127275   CONSTRAINT `fk_peer_review_file_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON
DELETE CASCADE
127276 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='AI审稿文件表';
127277
127278 -- 为审稿任务表添加文件外键
127279 ALTER TABLE `peer_review_tasks`
127280 ADD CONSTRAINT `fk_peer_review_task_file` FOREIGN KEY (`file_id`) REFERENCES `peer_review_fil
es` (`id`) ON DELETE SET NULL;
127281
127282 -- ============================================
127283 -- 反馈系统表
127284 -- ============================================
127285
127286 -- 用户反馈表
127287 CREATE TABLE IF NOT EXISTS `feedbacks` (
127288   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
127289   `email` VARCHAR(255) NOT NULL COMMENT '反馈者邮箱',
127290   `content` TEXT NOT NULL COMMENT '反馈内容',
127291   `attachments` TEXT COMMENT '附件列表（JSON格式，已废弃，改用feedback_attachments表）',
127292   `user_id` BIGINT UNSIGNED COMMENT '用户ID（登录用户，匿名为NULL）',
127293   `username` VARCHAR(100) COMMENT '用户名（登录用户）',
127294   `ip` VARCHAR(45) COMMENT '反馈者IP',
127295   `ip_location` VARCHAR(255) COMMENT 'IP归属地',
127296   `status` VARCHAR(20) NOT NULL DEFAULT 'pending' COMMENT '状态: pending=待处理, processed=已处理, re
plied=已回复, closed=已关闭',
127297   `reply_content` TEXT COMMENT '回复内容',
127298   `replied_at` DATETIME COMMENT '回复时间',
127299   `replied_by` VARCHAR(100) COMMENT '回复人',
```

```
127300   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127301   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
127302   PRIMARY KEY (`id`),
127303   INDEX `idx_email` (`email`),
127304   INDEX `idx_user_id` (`user_id`),
127305   INDEX `idx_status` (`status`),
127306   INDEX `idx_created_at` (`created_at`),
127307   INDEX `idx_status_created` (`status`, `created_at` DESC),
127308   CONSTRAINT `fk_feedback_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE S
ET NULL
127309 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户反馈表';
127310
127311 -- 反馈附件表
127312 CREATE TABLE IF NOT EXISTS `feedback_attachments` (
127313   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
127314   `upload_key` VARCHAR(36) NOT NULL COMMENT '上传批次key（UUID），用于关联未提交的附件',
127315   `feedback_id` BIGINT UNSIGNED COMMENT '关联的反馈ID（提交后填充）',
127316   `original_name` VARCHAR(255) NOT NULL COMMENT '原始文件名',
127317   `file_data` LONGBLOB NOT NULL COMMENT '文件二进制数据',
127318   `file_size` BIGINT UNSIGNED NOT NULL COMMENT '文件大小（字节）',
127319   `file_type` VARCHAR(20) NOT NULL COMMENT '文件类型: image=图片, file=文件',
127320   `mime_type` VARCHAR(100) NOT NULL COMMENT 'MIME类型',
127321   `is_used` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '是否已被使用（关联到反馈）',
127322   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127323   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
127324   PRIMARY KEY (`id`),
127325   INDEX `idx_upload_key` (`upload_key`),
127326   INDEX `idx_feedback_id` (`feedback_id`),
127327   INDEX `idx_is_used` (`is_used`),
127328   INDEX `idx_created_at` (`created_at`),
127329   CONSTRAINT `fk_attachment_feedback` FOREIGN KEY (`feedback_id`) REFERENCES `feedbacks` (`id
`) ON DELETE CASCADE
127330 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='反馈附件表';
127331
127332 -- 默认系统配置
127333 INSERT IGNORE INTO `system_configs` (`id`, `require_email_verification`) VALUES
127334 (1, TRUE);
127335   `extracted_text` LONGTEXT COMMENT '提取的文本内容',
127336   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127337
127338   PRIMARY KEY (`id`),
127339   INDEX `idx_user_id` (`user_id`),
127340   INDEX `idx_created_at` (`created_at`),
127341   CONSTRAINT `fk_peer_review_file_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON
DELETE CASCADE
127342 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='AI审稿文件表';
127343
127344 -- 为审稿任务表添加文件外键
127345 ALTER TABLE `peer_review_tasks`
127346 ADD CONSTRAINT `fk_peer_review_task_file` FOREIGN KEY (`file_id`) REFERENCES `peer_review_fil
es` (`id`) ON DELETE SET NULL;
127347
127348 -- ============================================
127349 -- 反馈系统表
127350 -- ============================================
127351
127352 -- 用户反馈表
```

```
127353 CREATE TABLE IF NOT EXISTS `feedbacks` (
127354   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
127355   `email` VARCHAR(255) NOT NULL COMMENT '反馈者邮箱',
127356   `content` TEXT NOT NULL COMMENT '反馈内容',
127357   `attachments` TEXT COMMENT '附件列表（JSON格式，已废弃，改用feedback_attachments表）',
127358   `user_id` BIGINT UNSIGNED COMMENT '用户ID（登录用户，匿名为NULL）',
127359   `username` VARCHAR(100) COMMENT '用户名（登录用户）',
127360   `ip` VARCHAR(45) COMMENT '反馈者IP',
127361   `ip_location` VARCHAR(255) COMMENT 'IP归属地',
127362   `status` VARCHAR(20) NOT NULL DEFAULT 'pending' COMMENT '状态: pending=待处理, processed=已处理, re
plied=已回复, closed=已关闭',
127363   `reply_content` TEXT COMMENT '回复内容',
127364   `replied_at` DATETIME COMMENT '回复时间',
127365   `replied_by` VARCHAR(100) COMMENT '回复人',
127366   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127367   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
127368   PRIMARY KEY (`id`),
127369   INDEX `idx_email` (`email`),
127370   INDEX `idx_user_id` (`user_id`),
127371   INDEX `idx_status` (`status`),
127372   INDEX `idx_created_at` (`created_at`),
127373   INDEX `idx_status_created` (`status`, `created_at` DESC),
127374   CONSTRAINT `fk_feedback_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE S
ET NULL
127375 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户反馈表';
127376
127377 -- 反馈附件表
127378 CREATE TABLE IF NOT EXISTS `feedback_attachments` (
127379   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
127380   `upload_key` VARCHAR(36) NOT NULL COMMENT '上传批次key（UUID），用于关联未提交的附件',
127381   `feedback_id` BIGINT UNSIGNED COMMENT '关联的反馈ID（提交后填充）',
127382   `original_name` VARCHAR(255) NOT NULL COMMENT '原始文件名',
127383   `file_data` LONGBLOB NOT NULL COMMENT '文件二进制数据',
127384   `file_size` BIGINT UNSIGNED NOT NULL COMMENT '文件大小（字节）',
127385   `file_type` VARCHAR(20) NOT NULL COMMENT '文件类型: image=图片, file=文件',
127386   `mime_type` VARCHAR(100) NOT NULL COMMENT 'MIME类型',
127387   `is_used` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '是否已被使用（关联到反馈）',
127388   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127389   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
127390   PRIMARY KEY (`id`),
127391   INDEX `idx_upload_key` (`upload_key`),
127392   INDEX `idx_feedback_id` (`feedback_id`),
127393   INDEX `idx_is_used` (`is_used`),
127394   INDEX `idx_created_at` (`created_at`),
127395   CONSTRAINT `fk_attachment_feedback` FOREIGN KEY (`feedback_id`) REFERENCES `feedbacks` (`id
`) ON DELETE CASCADE
127396 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='反馈附件表';
127397
127398 -- 默认系统配置
127399 INSERT IGNORE INTO `system_configs` (`id`, `require_email_verification`) VALUES
127400 (1, TRUE);
127401   `extracted_text` LONGTEXT COMMENT '提取的文本内容',
127402   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127403
127404   PRIMARY KEY (`id`),
127405   INDEX `idx_user_id` (`user_id`),
127406   INDEX `idx_created_at` (`created_at`),
```

```
127407   CONSTRAINT `fk_peer_review_file_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON
DELETE CASCADE
127408 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='AI审稿文件表';
127409
127410 -- 为审稿任务表添加文件外键
127411 ALTER TABLE `peer_review_tasks`
127412 ADD CONSTRAINT `fk_peer_review_task_file` FOREIGN KEY (`file_id`) REFERENCES `peer_review_fil
es` (`id`) ON DELETE SET NULL;
127413
127414 -- ============================================
127415 -- 反馈系统表
127416 -- ============================================
127417
127418 -- 用户反馈表
127419 CREATE TABLE IF NOT EXISTS `feedbacks` (
127420   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
127421   `email` VARCHAR(255) NOT NULL COMMENT '反馈者邮箱',
127422   `content` TEXT NOT NULL COMMENT '反馈内容',
127423   `attachments` TEXT COMMENT '附件列表（JSON格式，已废弃，改用feedback_attachments表）',
127424   `user_id` BIGINT UNSIGNED COMMENT '用户ID（登录用户，匿名为NULL）',
127425   `username` VARCHAR(100) COMMENT '用户名（登录用户）',
127426   `ip` VARCHAR(45) COMMENT '反馈者IP',
127427   `ip_location` VARCHAR(255) COMMENT 'IP归属地',
127428   `status` VARCHAR(20) NOT NULL DEFAULT 'pending' COMMENT '状态: pending=待处理, processed=已处理, re
plied=已回复, closed=已关闭',
127429   `reply_content` TEXT COMMENT '回复内容',
127430   `replied_at` DATETIME COMMENT '回复时间',
127431   `replied_by` VARCHAR(100) COMMENT '回复人',
127432   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127433   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
127434   PRIMARY KEY (`id`),
127435   INDEX `idx_email` (`email`),
127436   INDEX `idx_user_id` (`user_id`),
127437   INDEX `idx_status` (`status`),
127438   INDEX `idx_created_at` (`created_at`),
127439   INDEX `idx_status_created` (`status`, `created_at` DESC),
127440   CONSTRAINT `fk_feedback_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE S
ET NULL
127441 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户反馈表';
127442
127443 -- 反馈附件表
127444 CREATE TABLE IF NOT EXISTS `feedback_attachments` (
127445   `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
127446   `upload_key` VARCHAR(36) NOT NULL COMMENT '上传批次key（UUID），用于关联未提交的附件',
127447   `feedback_id` BIGINT UNSIGNED COMMENT '关联的反馈ID（提交后填充）',
127448   `original_name` VARCHAR(255) NOT NULL COMMENT '原始文件名',
127449   `file_data` LONGBLOB NOT NULL COMMENT '文件二进制数据',
127450   `file_size` BIGINT UNSIGNED NOT NULL COMMENT '文件大小（字节）',
127451   `file_type` VARCHAR(20) NOT NULL COMMENT '文件类型: image=图片, file=文件',
127452   `mime_type` VARCHAR(100) NOT NULL COMMENT 'MIME类型',
127453   `is_used` BOOLEAN NOT NULL DEFAULT FALSE COMMENT '是否已被使用（关联到反馈）',
127454   `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
127455   `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
127456   PRIMARY KEY (`id`),
127457   INDEX `idx_upload_key` (`upload_key`),
127458   INDEX `idx_feedback_id` (`feedback_id`),
127459   INDEX `idx_is_used` (`is_used`),
```

```
127460   INDEX `idx_created_at` (`created_at`),
127461   CONSTRAINT `fk_attachment_feedback` FOREIGN KEY (`feedback_id`) REFERENCES `feedbacks` (`id
`) ON DELETE CASCADE
127462 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='反馈附件表';
127463
127464 -- 默认系统配置
127465 INSERT IGNORE INTO `system_configs` (`id`, `require_email_verification`) VALUES
127466 (1, TRUE);
127467 -- ============================================
127468 -- 数据库迁移：为 peer_review_tasks 添加 file_name 字段
127469 -- ============================================
127470 SET @exist := (SELECT COUNT(*) FROM INFORMATION_SCHEMA.COLUMNS
127471             WHERE TABLE_SCHEMA = DATABASE()
127472             AND TABLE_NAME = 'peer_review_tasks'
127473             AND COLUMN_NAME = 'file_name');
127474 SET @sql := IF(@exist = 0,
127475     'ALTER TABLE `peer_review_tasks` ADD COLUMN `file_name` VARCHAR(500) COMMENT "上传的文件名" A
FTER `file_id`',
127476     'SELECT 1');
127477 PREPARE stmt FROM @sql;
127478 EXECUTE stmt;
127479 DEALLOCATE PREPARE stmt;
```

==========

文 档 结 束

==========

软件名称：Scholay学术平台
版本号：V1.0
著作权人：成都恩特罐罐文化科技有限公司
统一社会信用代码：91510100MAEEHRNE26

Copyright (c) 2025 成都恩特罐罐文化科技有限公司
All Rights Reserved. 保留所有权利。

==========